Companion to George Grätzer's

Vibe Computing for Mathematicians

Python Programs

Complete programs from the book and exercise companion

Contents

How to Use this Companion vii

1 Mathematics First 1
1.1 Main-Book Programs 1
ch01_sum_five.py 1
ch01_sum_ten.py 1
ch01_petersen.py 1
ch01_primes.py 2
ch01_euler.py 3
ch01_fibonacci.py 4
1.2 Exercise Programs 4
ch01_examples.py 5

2 Arithmetic Investigations 7
2.1 Main-Book Programs 7
ch02_prime_gaps.py 7
ch02_divisors.py 8
ch02_euclid.py 8
ch02_residues.py 9
ch02_pseudoprime.py 9
2.2 Exercise Programs 9
ch02_examples.py 9

3 Conjectures and Tests 11
3.1 Main-Book Programs 11
ch03_odd_sums.py 11
ch03_interpolation.py 11
ch03_gcd_properties.py 12
ch03_mersenne.py 12
3.2 Exercise Programs 13
ch03_examples.py 13

4 Trustworthy Mathematical Answers 15
4.1 Main-Book Programs 15
ch04_divisor_repair.py 15
ch04_inversions.py 16
ch04_euclid.py 17

ch04_sorted_copy.py . . . 17
ch04_binomial.py . . . 18
4.2 Exercise Programs . . . 18
ch04_examples.py . . . 18

5 Exact and Approximate Methods . . . 21
5.1 Main-Book Programs . . . 21
ch05_symbolic.py . . . 21
ch05_tolerances.py . . . 21
ch05_cancellation.py . . . 22
ch05_bisection.py . . . 22
ch05_quadratic.py . . . 23
5.2 Exercise Programs . . . 24
ch05_examples.py . . . 24

6 Pictures, Tables, and Evidence . . . 27
6.1 Main-Book Programs . . . 27
ch06_error_plot.py . . . 27
ch06_prime_gaps.py . . . 28
ch06_convergence.py . . . 29
ch06_csv.py . . . 30
6.2 Exercise Programs . . . 30
ch06_examples.py . . . 31

7 Sequences and Linear Algebra . . . 33
7.1 Main-Book Programs . . . 33
ch07_recurrence.py . . . 33
ch07_modular_period.py . . . 33
ch07_generating_function.py . . . 34
ch07_exact_rank.py . . . 34
ch07_linear_system.py . . . 35
ch07_vandermonde.py . . . 35
ch07_coordinates.py . . . 36
ch07_numerical_rank.py . . . 36
7.2 Exercise Programs . . . 36
ch07_examples.py . . . 37

8 Dynamics and Randomness . . . 39
8.1 Main-Book Programs . . . 39
ch08_random_walk.py . . . 39
ch08_logistic.py . . . 40
8.2 Exercise Programs . . . 41
ch08_examples.py . . . 41

9 Graphs and Networks . . . 43
9.1 Main-Book Programs . . . 43
ch09_paths.py . . . 43
ch09_colouring.py . . . 44
ch09_spanning_tree.py . . . 44
ch09_hamilton.py . . . 45
9.2 Exercise Programs . . . 46

ch09_examples.py . . . 46

10 Search and Ordered Structures . . . 49
10.1 Main-Book Programs . . . 49
ch10_enumeration.py . . . 49
ch10_permutations.py . . . 49
ch10_coin_change.py . . . 50
ch10_order_tools.py . . . 50
ch10_lattice.py . . . 52
ch10_distributivity.py . . . 54
ch10_ideal_lattice.py . . . 56
ch10_m3.py . . . 57
10.2 Exercise Programs . . . 57
ch10_examples.py . . . 58

11 Testing and Proof . . . 59
11.1 Main-Book Programs . . . 59
ch11_gcd_invariant.py . . . 59
ch11_coloring_checker.py . . . 59
11.2 Exercise Programs . . . 60
ch11_examples.py . . . 60

12 Reproducible Investigations . . . 63
12.1 Main-Book Programs . . . 63
ch12_seeded_pairs.py . . . 63
ch12_operations.py . . . 64
ch12_capstone.py . . . 65
12.2 Exercise Programs . . . 66
ch12_examples.py . . . 66

A Python: An Optional Introduction . . . 69
A.1 Main-Book Programs . . . 69
appendix_a_first_run.py . . . 69
hello.py . . . 69
appendix_a_numbers.py . . . 70
appendix_a_loops.py . . . 71
appendix_a_functions.py . . . 72
appendix_a_copying.py . . . 75
appendix_a_exceptions.py . . . 76
appendix_a_module.py . . . 76
A.2 Exercise Programs . . . 77
appa_examples.py . . . 77

B Verification and Reproduction Utilities . . . 83
B.1 Exercise Support . . . 83
ch10_examples 2.py . . . 83
divisor_tool.py . . . 84
make_tables.py . . . 85
ordered_set_analyzer.py . . . 86
prepare_examples.py . . . 88
run_checks.py . . . 89

verify_solutions.py 89
B.2 Collection and Build Tools 94
build_books.py 94
check_book.py 95
synchronize.py 98
test_synchronization.py 104

How to Use this Companion

Use this companion when you want to find, run, or read a program from the book. It prints every supplied Python file in full; editable copies are included in the package. The main book develops the mathematics, and the exercise companion gives hints and solutions. You can ask ChatGPT to run a named file and explain the output, whether or not you choose to study the code.

The collection contains 64 complete main-book programs, 13 complete exercise chapter programs, the two capstone modules, and all extraction, verification, table-generation, synchronization, and build utilities. When a program continues across several listings in the main book, its parts are joined here in reading order. The two longer lattice routines appear here in full and require a nonempty universe. The Appendix A editor example is supplied separately as `main_programs/hello.py`. The quotation-mark example in Appendix A deliberately contains an error, and its two package-installation commands are meant for a terminal. These are therefore not included as executable Python files.

Running a program

Use Python 3.12 or later. The file `requirements.txt` lists the package versions. Each verification record also reports the interpreter and package versions used for that run. From this companion's directory, install the requirements with `python3 -m pip install -r requirements.txt`. On Windows, use `py` where these instructions say `python3`. These are terminal commands. Run a selected file, for example, `python3 main_programs/ch01_euler.py`. Run complete chapter files such as `python3 exercises/programs/ch04_examples.py`; their later examples may use earlier definitions. Run without `-O`, so assertions remain active. Graphical examples may open plot windows during an interactive run; the verification commands select a noninteractive plotting backend.

Repeating the checks

Run `python3 tools/check_book.py` for all 64 main-book files and the Euler and lattice boundary checks, including empty-universe rejection. It also runs the saved-record experiment twice in separate processes in the same folder, verifies that its record is unchanged, and checks that a deliberately altered sample is rejected. It saves the output from the separate processes, the environment details, and the source hashes in `results/book_checks.json`. It also runs `ch12_operations.py` and writes `results/three_operations.json`: the complete frequency distribution, one table for each of

the 28 attained counts, and every failing triple for each retained table. A second dictionary-based calculation checks each witness. The sum of the frequencies is 19,683 and the zero-failure count is 113.

Run `python3 exercises/programs/run_checks.py` for the 13 exercise chapter files and both capstones. Run `python3 exercises/programs/verify_solutions.py` to regenerate the exercise tables, finite searches, and figures. Its retained record is `exercises/results/verified_results.json`. Then run `python3 exercises/programs/make_tables.py` to refresh the embedded LaTeX tables. The commands can take a few minutes.

The complete manuscript package includes the figures used in the main book. The manifest identifies the related programs. Some of those programs produce data or display plots without saving the figures as they appear in the book. The exercise verification program regenerates all four exercise figures. Integer and finite-enumeration checks are exact; floating-point last digits and long chaotic orbits can depend on the environment.

Maintaining the sources

In the complete package, edit `../Vibe_Math.tex` and `../Solutions_Companion/Exercises_Help.tex`. For the two external Chapter 10 programs, edit their files directly in `main_programs/`. Run `python3 tools/synchronize.py` to refresh the remaining main-book files, the exercise snapshot, the current exercise inventory, this source, and `program_manifest.json`. Use `python3 tools/synchronize.py --check` to detect stale generated source files without changing them. Run `python3 tools/test_synchronization.py` for the extraction regression tests. The editable catalogue introduction is `tools/catalogue_frontmatter.tex`. Program and exercise counts are derived from the sources, not fixed constants. Run `python3 tools/build_books.py` to compile all three books. The copy of the exercise companion included here can be used on its own. To revise it, edit the original in `../Solutions_Companion/` and then synchronize. The manifest contains a hash for every Python file and the main-book source locations. This PDF reads the actual files with `lstinputlisting`, so its listings and the downloadable programs use the same text.

Chapter 1
Mathematics First

1.1 Main-Book Programs

Source: A First Mathematical Conversation.

ch01_sum_five.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

print(1 + 2 + 3 + 4 + 5)
```

Source: A First Mathematical Conversation.

ch01_sum_ten.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

print(1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10)
```

Source: Four Experiments.

ch01_petersen.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""
```

```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.petersen_graph()

from math import cos, sin, pi
pos = {}
for k in range(5):
    angle = pi / 2 - 2 * pi * k / 5
    pos[k] = (2 * cos(angle), 2 * sin(angle))
    pos[k + 5] = (cos(angle), sin(angle))

nx.draw(
    G,
    pos,
    with_labels=True,
    node_color="lightblue",
    node_size=900,
    font_size=14,
    font_weight="bold",
    edge_color="black"
)

plt.title("Petersen Graph")
plt.show()

print("Connected:", nx.is_connected(G))
```

Source: Four Experiments.

ch01_primes.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

def is_prime(n):
    if n < 2:
        return False

    divisor = 2
    while divisor * divisor <= n:
        if n % divisor == 0:
            return False
        divisor += 1

    return True


primes = []
number = 2

while len(primes) < 10000:
    if is_prime(number):
        primes.append(number)
    number += 1
```

```
print("Largest prime:", primes[-1])

print("Last ten primes:")

for p in primes[-10:]:
    print(p)
```

Source: Verification and Explanation.

ch01_euler.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

import networkx as nx

def test_euler(name, G):
    if G.is_directed() or G.is_multigraph():
        raise ValueError("a simple undirected graph is required")
    if nx.number_of_selfloops(G) != 0:
        raise ValueError("loops are not permitted")
    if len(G) == 0 or not nx.is_connected(G):
        raise ValueError("a nonempty connected graph is required")
    planar, embedding = nx.check_planarity(G)

    if not planar:
        print(name, "is not planar")
        return

    visited = set()
    faces = []

    for u, v in embedding.edges():
        if (u, v) not in visited:
            face = embedding.traverse_face(u, v, visited)
            faces.append(face)

    vertices = G.number_of_nodes()
    edges = G.number_of_edges()
    number_of_faces = len(faces) if edges else 1
    assert vertices - edges + number_of_faces == 2

    print(name)
    print("v =", vertices)
    print("e =", edges)
    print("f =", number_of_faces)
    print("v - e + f =", vertices - edges + number_of_faces)
    print()
    return vertices, edges, number_of_faces


graphs = {
    "Cycle C6": nx.cycle_graph(6),
    "Wheel W7": nx.wheel_graph(7),
```

```
    "Complete graph K4": nx.complete_graph(4)
}

for name, graph in graphs.items():
    test_euler(name, graph)

assert test_euler("Single vertex", nx.empty_graph(1)) == (1, 0, 1)
disconnected = nx.disjoint_union(nx.cycle_graph(3), nx.cycle_graph(3))
try:
    test_euler("Two disjoint triangles", disconnected)
except ValueError:
    print("Disconnected input rejected")
else:
    raise AssertionError("disconnected input was accepted")
```

Source: Verification and Explanation.

ch01_fibonacci.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

import math

fibonacci = [0, 1]

while len(fibonacci) < 30:
    fibonacci.append(fibonacci[-1] + fibonacci[-2])

phi = (1 + math.sqrt(5)) / 2

print("Golden ratio:", phi)
print()

for n in range(1, len(fibonacci) - 1):
    ratio = fibonacci[n + 1] / fibonacci[n]
    error = abs(ratio - phi)

    print(
        f"n={n:2d}",
        f"ratio={ratio:.12f}",
        f"error={error:.12e}"
    )
```

1.2 Exercise Programs

ch01_examples.py

```
"""Printed listings from ch01. Run this complete file."""
import matplotlib
matplotlib.use("Agg")

def sum_cubes(n):
    if type(n) is not int or n < 0:
        raise ValueError("expected integer >= 0")
    total = 0
    for k in range(1, n + 1):
        total += k**3
    return total

assert [sum_cubes(n) for n in range(4)] == [0, 1, 9, 36]
print(sum_cubes(3))
for n in range(11):
    assert sum_cubes(n) == sum(range(n + 1))**2
    print(n, sum_cubes(n), sum(range(n + 1))**2)

import networkx as nx
import matplotlib.pyplot as plt
G = nx.petersen_graph()
fig, axes = plt.subplots(1, 2, figsize=(8, 4))
for ax, positions in zip(axes, [nx.circular_layout(G),
                                nx.spring_layout(G, seed=17)]):
    nx.draw_networkx(G, pos=positions, ax=ax,
                     node_color="white", edgecolors="black",
                     node_size=300, font_size=9)
    ax.set_axis_off()
fig.tight_layout()
plt.close(fig)
assert len(G) == 10 and G.number_of_edges() == 15
assert sorted(dict(G.degree()).values()) == [3] * 10

from math import isqrt

def is_prime(n):
    if type(n) is not int:
        raise ValueError("expected integer")
    return n >= 2 and all(n % d != 0
                          for d in range(2, isqrt(n) + 1))

primes = [n for n in range(2, 200) if is_prime(n)]
assert len(primes) == 46

f = [0, 1]
for _ in range(19):
    f.append(f[-1] + f[-2])
ratios = [(n, f[n] / f[n-1]) for n in range(5, 21)]
```

Chapter 2
Arithmetic Investigations

2.1 Main-Book Programs

Source: Prime-Gap Investigation.

ch02_prime_gaps.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

def is_prime(n):
    if n < 2:
        return False
    divisor = 2
    while divisor * divisor <= n:
        if n % divisor == 0:
            return False
        divisor += 1
    return True

primes = []

for number in range(2, 1000):
    if is_prime(number):
        primes.append(number)

gaps = []

for i in range(len(primes) - 1):
    gaps.append(primes[i + 1] - primes[i])

print("First 30 prime gaps:")
print(gaps[:30])

assert primes[:6] == [2, 3, 5, 7, 11, 13]
assert all(primes[i] < primes[i + 1]
           for i in range(len(primes) - 1))
assert gaps == [primes[i + 1] - primes[i]
```

```
                for i in range(len(primes) - 1)]

print("first primes:", primes[:6])
print("first gaps:", gaps[:5])
```

Source: Divisors and Algorithms.

ch02_divisors.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

def divisors(n):
    result = []

    for d in range(1, n + 1):
        if n % d == 0:
            result.append(d)

    return result


for n in range(1, 51):
    ds = divisors(n)
    print(n, ds, "number of divisors =", len(ds))
```

Source: Divisors and Algorithms.

ch02_euclid.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

def euclid(a, b):
    a, b = abs(a), abs(b)
    while b != 0:
        q = a // b
        r = a % b

        print(a, "=", q, "*", b, "+", r)

        a, b = b, r

    return a


g = euclid(1234567890, 987654321)

print("gcd =", g)
```

```
from math import gcd
assert gcd(1234567890, 987654321) == 9
assert 1234567890 % 9 == 987654321 % 9 == 0
```

Source: Remainders and Primes.

ch02_residues.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

def power_residues(power, modulus):
    return sorted({
        pow(a, power, modulus)
        for a in range(modulus)
    })

print(power_residues(2, 8))
```

Source: Remainders and Primes.

ch02_pseudoprime.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

def passes_base_two_test(n):
    return n > 1 and pow(2, n - 1, n) == 1

print("341 passes:", passes_base_two_test(341))
```

2.2 Exercise Programs

ch02_examples.py

```
"""Printed listings from ch02. Run this complete file."""

def proper_divisors(n):
    if type(n) is not int or n < 1:
        raise ValueError("expected positive integer")
    return [d for d in range(1, n) if n % d == 0]

assert proper_divisors(1) == []
assert proper_divisors(12) == [1, 2, 3, 4, 6]
```

```
print(sum(proper_divisors(12)))
for n in range(1, 31):
    ds = proper_divisors(n)
    print(n, ds, sum(ds))
assert [n for n in range(1, 31) if sum(proper_divisors(n)) == n] == [6, 28]

answers = []
n = 1
while len(answers) < 10:
    if sum(n % d == 0 for d in range(1, n + 1)) == 6:
        answers.append(n)
    n += 1
assert answers == [12,18,20,28,32,44,45,50,52,63]

assert 11 * 31 == 341
assert pow(2, 340, 341) == 1
```

Chapter 3
Conjectures and Tests

3.1 Main-Book Programs

Source: Observation and Conjecture.

ch03_odd_sums.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

def odd_partial_sums(count):
    total = 0
    values = []
    for n in range(1, count + 1):
        total += 2*n - 1
        values.append(total)
    return values

print(odd_partial_sums(5))
```

Source: Observation and Conjecture.

ch03_interpolation.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

import sympy as sp

n = sp.symbols("n")
values = [1, 2, 4, 8, 16, 100]
p = sp.expand(sp.interpolate(list(enumerate(values)), n))
print(p)
```

```
for k, value in enumerate(values):
    assert p.subs(n, k) == value
    print(f"p({k}) = {p.subs(n, k)}")
```

Source: Breaking the Claim.

ch03_gcd_properties.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

import math
import random

rng = random.Random(1729)
for _ in range(10000):
    a = rng.randint(1, 10**8)
    b = rng.randint(1, 10**8)
    g = math.gcd(a, b)
    assert g == math.gcd(b, a)
    assert a % g == 0 and b % g == 0
    assert g == math.gcd(a - b, b)

print("10,000 random gcd checks passed")
```

Source: Repairing a Mersenne Conjecture.

ch03_mersenne.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

from math import isqrt

def smallest_factor(number):
    if number < 2:
        return None
    for divisor in range(2, isqrt(number) + 1):
        if number % divisor == 0:
            return divisor
    return None

def is_prime(number):
    return number >= 2 and smallest_factor(number) is None

def first_composite_mersenne(limit):
    for exponent in range(2, limit + 1):
        if not is_prime(exponent):
            continue
        value = 2**exponent - 1
```

```
        factor = smallest_factor(value)
        if factor is not None:
            cofactor = value // factor
            assert factor * cofactor == value
            return exponent, value, factor, cofactor
    return None

assert not is_prime(1)
assert is_prime(2)
assert not is_prime(9)
assert is_prime(13)
assert not is_prime(49)
print(first_composite_mersenne(31))
```

3.2 Exercise Programs

ch03_examples.py

```
"""Printed listings from ch03. Run this complete file."""

def first_divisible(values, divisor):
    if type(divisor) is not int or divisor <= 0:
        raise ValueError("expected positive integer divisor")
    if any(type(x) is not int for x in values):
        raise ValueError("expected integer entries")
    for x in values:
        if x % divisor == 0:
            return x
    return None

assert first_divisible([], 3) is None
assert first_divisible([5, 0, 6], 3) == 0
answer = first_divisible([5, 0, 6], 3)
assert answer is not None
for values in [[], [1, 2, 4], [0, 1, 3]]:
    answer = first_divisible(values, 3)
    print("no such entry" if answer is None else answer)
assert first_divisible([1, 2, 4], 3) is None
assert first_divisible([0, 1, 3], 3) == 0

from math import isqrt

def square_root_test(n):
    if type(n) is not int or n < 0:
        raise ValueError("expected integer >= 0")
    r = isqrt(n)
    return r*r == n

def square_search_test(n):
    if type(n) is not int or n < 0:
        raise ValueError("expected integer >= 0")
    left, right = 0, n + 1
    while left < right:
        mid = (left + right) // 2
```

```
        if mid*mid < n:
            left = mid + 1
        else:
            right = mid
    return left*left == n

assert all(square_root_test(n) == square_search_test(n)
           for n in range(10001))

from math import isqrt

def factors(n):
    if type(n) is not int or n < 2:
        raise ValueError("expected integer >= 2")
    result = []
    d = 2
    while d*d <= n:
        while n % d == 0:
            result.append(d)
            n //= d
        d += 1
    if n > 1:
        result.append(n)
    return result

prime_exponents = [p for p in range(2, 32)
                   if factors(p) == [p]]
for p in prime_exponents:
    n = 2**p - 1
    fs = factors(n)
    from math import prod
    assert prod(fs) == n
    assert all(factors(q) == [q] for q in fs)
```

Chapter 4
Trustworthy Mathematical Answers

4.1 Main-Book Programs

Source: A Wrong Answer That Teaches Something.

ch04_divisor_repair.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

def divisor_count(n):
    count = 0
    d = 1
    while d * d < n:
        if n % d == 0:
            count += 2
        d += 1
    return count

print("n=12:", divisor_count(12))
print("n=9:", divisor_count(9))

from math import isqrt

def divisor_count(n):
    if not isinstance(n, int) or isinstance(n, bool) or n <= 0:
        raise ValueError("n must be a positive integer")

    count = 0
    for d in range(1, isqrt(n) + 1):
        if n % d == 0:
            count += 1 if d * d == n else 2
    return count

def divisor_count_reference(n):
    if not isinstance(n, int) or isinstance(n, bool) or n <= 0:
        raise ValueError("n must be a positive integer")
    return sum(1 for d in range(1, n + 1) if n % d == 0)
```

```
assert divisor_count(1) == 1
assert divisor_count(9) == 3
assert divisor_count(13) == 2
assert divisor_count(12) == 6

for n in range(1, 10001):
    assert divisor_count(n) == divisor_count_reference(n)

print("all comparisons passed")

from math import prod

def divisors(n):
    small = [d for d in range(1, isqrt(n) + 1) if n % d == 0]
    large = [n // d for d in reversed(small) if d * d != n]
    return small + large

for n in range(1, 1001):
    ds = divisors(n)
    assert ds == sorted(set(ds))
    assert ds[0] == 1 and ds[-1] == n
    assert all(n % d == 0 for d in ds)
    assert len(ds) == divisor_count(n)
    expected = n ** (len(ds) // 2)
    if len(ds) % 2:
        expected *= isqrt(n)
    assert prod(ds) == expected

print("divisor checks passed for 1 through 1,000")

record = 0
for n in range(1, 101):
    count = divisor_count(n)
    if count > record:
        record = count
        print(n, count)
```

Source: What Is Being Counted?.

ch04_inversions.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

def inversions(p):
    n = len(p)
    if any(type(value) is not int for value in p):
        raise ValueError("entries must be integers")
    if sorted(p) != list(range(n)):
        raise ValueError("expected permutation 0,...,n-1")

    count = 0
    for i in range(n):
        for j in range(i + 1, n):
```

```
            if p[i] > p[j]:
                count += 1
    return count

assert inversions([]) == 0
assert inversions([0, 1, 2]) == 0
assert inversions([2, 0, 1]) == 2
assert inversions([2, 1, 0]) == 3
print(inversions([2, 0, 3, 1]))
```

Source: What Is Being Counted?.

ch04_euclid.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

def gcd_euclid(a, b):
    a, b = abs(a), abs(b)

    while b != 0:
        a, b = b, a % b

    return a

print(gcd_euclid(84, 30))
```

Source: Boundaries and Independent Checks.

ch04_sorted_copy.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

def sorted_copy(values):
    answer = list(values)
    answer.sort()
    return answer

data = [3, 1, 2]
answer = sorted_copy(data)

assert data == [3, 1, 2]
assert answer == [1, 2, 3]

print("original:", data)
print("sorted copy:", answer)
```

Source: Boundaries and Independent Checks.

ch04_binomial.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

def binomial_row(n):
    if n < 0:
        raise ValueError("n must be nonnegative")

    row = [1]
    for k in range(1, n + 1):
        row.append(row[-1] * (n - k + 1) // k)

    assert len(row) == n + 1
    assert sum(row) == 2**n
    assert row == row[::-1]
    return row

print(binomial_row(5))
```

4.2 Exercise Programs

ch04_examples.py

```
"""Printed listings from ch04. Run this complete file."""

def inversions(p):
    n = len(p)
    if any(type(x) is not int for x in p):
        raise ValueError("entries must be integers")
    if sorted(p) != list(range(n)):
        raise ValueError("expected permutation 0,...,n-1")
    count = 0
    for i in range(n):
        for j in range(i + 1, n):
            if p[i] > p[j]:
                count += 1
    return count

def inversions_compact(p):
    if any(type(x) is not int for x in p):
        raise ValueError("entries must be integers")
    if sorted(p) != list(range(len(p))):
        raise ValueError("expected permutation")
    return sum(p[i] > p[j] for i in range(len(p))
               for j in range(i + 1, len(p)))
```

```
for bad in [[0, 0], [-1, 0], [False]]:
    try:
        inversions(bad)
    except ValueError:
        pass
    else:
        raise AssertionError("invalid input accepted")
p = [2, 0, 3, 1]
before = p.copy()
assert inversions(p) == 3 and p == before
from itertools import permutations
for n in range(8):
    for p in permutations(range(n)):
        assert inversions(p) == inversions_compact(p)
assert inversions((2, 0, 1)) == 2
assert inversions((2, 1, 0)) == 3
for n in range(11):
    assert inversions(tuple(range(n))) == 0
    assert inversions(tuple(reversed(range(n)))) == n*(n-1)//2

from math import isqrt

def divisor_count(n):
    if type(n) is not int or n < 1:
        raise ValueError("expected positive integer")
    count = 0
    for d in range(1, isqrt(n) + 1):
        if n % d == 0:
            count += 1 if d*d == n else 2
    return count

assert divisor_count(9) == 3
assert divisor_count(1) == 1
for bad in [0, -1, 3.5, True]:
    try:
        divisor_count(bad)
    except ValueError:
        pass
    else:
        raise AssertionError("invalid input accepted")

def divisor_count_reference(n):
    if type(n) is not int or n < 1:
        raise ValueError("expected positive integer")
    return sum(n % d == 0 for d in range(1, n + 1))

assert all(divisor_count(n) == divisor_count_reference(n)
           for n in range(1, 10001))
```

Chapter 5
Exact and Approximate Methods

5.1 Main-Book Programs

Source: Exact Form and Assumptions.

ch05_symbolic.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

import sympy as sp

x = sp.symbols("x")
numerator = x**4 - 1
expression = numerator / (x**2 - 1)

print(sp.factor(numerator))
print(sp.cancel(expression))

k, n = sp.symbols("k n", integer=True, positive=True)
answer = sp.summation(k**2, (k, 1, n))
print(sp.factor(answer))
```

Source: Floating Point and Stability.

ch05_tolerances.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

import math

x = 0.1 + 0.2
```

```
y = 0.3

assert math.isclose(x, y, rel_tol=1e-12, abs_tol=1e-15)

print("exactly equal:", x == y)
print("close:", math.isclose(x, y, rel_tol=1e-12, abs_tol=1e-15))
```

Source: Floating Point and Stability.

ch05_cancellation.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

import math

for x in [1e-4, 1e-8, 1e-12, 1e-16]:
    direct = math.sqrt(1 + x) - 1
    stable = x / (math.sqrt(1 + x) + 1)
    print(x, direct, stable)
```

Source: Certificates and Precision.

ch05_bisection.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

import math

def bisect(f, left, right, tolerance=1e-12):
    if not all(map(math.isfinite, (left, right, tolerance))):
        raise ValueError("endpoints and tolerance must be finite")
    if left >= right or tolerance <= 0:
        raise ValueError("require left < right and tolerance > 0")

    def value(x):
        y = f(x)
        if not math.isfinite(y):
            raise ValueError("nonfinite function value")
        return y

    f_left, f_right = value(left), value(right)
    if f_left == 0:
        return left, (left, left)
    if f_right == 0:
        return right, (right, right)
    if (f_left < 0) == (f_right < 0):
        raise ValueError("endpoint signs must differ")
```

```
    while right - left > tolerance:
        middle = left / 2 + right / 2
        if middle == left or middle == right:
            raise ArithmeticError("tolerance below float resolution")
        f_middle = value(middle)
        if f_middle == 0:
            return middle, (middle, middle)
        if (f_left < 0) == (f_middle < 0):
            left, f_left = middle, f_middle
        else:
            right, f_right = middle, f_middle
    return left / 2 + right / 2, (left, right)

root, bracket = bisect(lambda x: x*x - 2, 1, 2)
print(f"root = {root:.12f}")
print(f"bracket width = {bracket[1] - bracket[0]:.3e}")

scaled = lambda x: 1e-200 * (x*x - 2)
_, interval = bisect(scaled, 1, 2)
assert scaled(interval[0]) < 0 < scaled(interval[1])
assert interval[0] <= math.sqrt(2) <= interval[1]
print("scaled-function regression passed")
```

Source: A Quadratic Loses a Root.

ch05_quadratic.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

import math

def quadratic_roots(B):
    if not math.isfinite(B) or B < 2:
        raise ValueError("B must be finite and at least 2")
    # Scaling avoids both B*B and B + sqrt(B*B - 4).
    t = 2.0 / B
    scaled = math.sqrt((1.0 - t) * (1.0 + t))
    large = (B / 2.0) * (1.0 + scaled)
    small_direct = (B - B * scaled) / 2.0
    small_stable = 1.0 / large
    return large, small_direct, small_stable

for B in [1e4, 1e8, 1e12, 1e200]:
    large, direct, stable = quadratic_roots(B)
    print(f"B = {B:.0e}")
    for name, x in [("direct", direct), ("stable", stable)]:
        residual = (x - B) * x + 1.0
        print(f" {name}: small={x:.6g}")
        print(f" product={x * large:.6g}",
              f"residual={residual:.3e}")
    assert math.isfinite(large) and stable > 0
    assert math.isclose(large * stable, 1.0, rel_tol=1e-14)
assert quadratic_roots(2.0) == (1.0, 1.0, 1.0)
```

```
from decimal import Decimal, localcontext

def decimal_small_root(text, precision=60):
    with localcontext() as context:
        context.prec = precision
        B = Decimal(text)
        root = (B * B - Decimal(4)).sqrt()
        return Decimal(2) / (B + root)

for digits in [30, 45, 60]:
    value = decimal_small_root("1e12", digits)
    print(digits, format(value, ".26E"))
```

5.2 Exercise Programs

ch05_examples.py

```
"""Printed listings from ch05. Run this complete file."""

from fractions import Fraction
import math
exact = Fraction(1, 3)
approx = 1 / 3
assert 3 * exact == 1
assert 3 * approx == 1.0
assert Fraction(approx) != exact
assert Fraction(approx) - exact == -Fraction(1, 54043195528445952)
assert math.isclose(approx, float(exact),
                    rel_tol=1e-12, abs_tol=1e-15)

assert not math.isclose(0.0, 1e-14, rel_tol=1e-6, abs_tol=0.0)
assert math.isclose(0.0, 1e-14, rel_tol=1e-6, abs_tol=1e-12)

import sympy as sp
x = sp.symbols('x', real=True)
candidates = sp.solve(x*x - x - 2, x)
solutions = [c for c in candidates
             if sp.simplify(sp.sqrt(c + 2) - c) == 0]
assert solutions == [2]

import math
for x in [1e-4, 1e-8, 1e-12, 1e-16, 1e-20]:
    direct = math.sqrt(1 + x) - 1
    stable = x / (math.sqrt(1 + x) + 1)
    assert stable > 0

assert math.isclose(7e-13, 0.0, rel_tol=0, abs_tol=1e-12)
assert not math.isclose(2e-12, 0.0, rel_tol=0, abs_tol=1e-12)
assert math.isclose(1e6 + 0.0005, 1e6,
                    rel_tol=1e-9, abs_tol=1e-12)
assert not math.isclose(1e6 + 0.002, 1e6,
                        rel_tol=1e-9, abs_tol=1e-12)

def negative_quadratic_roots(B):
```

```
    if not math.isfinite(B) or B < 2:
        raise ValueError("finite B >= 2 required")
    t = 2.0 / B
    s = math.sqrt((1 - t) * (1 + t))
    large = -(B / 2) * (1 + s)
    direct = (-B + B*s) / 2
    stable = 1 / large
    return large, direct, stable

assert negative_quadratic_roots(2) == (-1, -1, -1)
for B in [1e4, 1e8, 1e12, 1e200]:
    large, direct, small = negative_quadratic_roots(B)
    assert large < 0 and small < 0
    assert math.isclose(large*small, 1, rel_tol=1e-14)

scan = [(k, negative_quadratic_roots(10.0**k))
        for k in range(1, 17)]
first_zero = next(k for k, roots in scan if roots[1] == 0)
assert first_zero == 10  # recorded binary64 environment

from decimal import Decimal, localcontext
for precision in [30, 60]:
    with localcontext() as ctx:
        ctx.prec = precision
        x = Decimal("1e-20")
        direct = (1 + x).sqrt() - 1
        stable = x / ((1 + x).sqrt() + 1)
        assert direct > 0 and stable > 0
```

Chapter 6
Pictures, Tables, and Evidence

6.1 Main-Book Programs

Source: Mathematical Figures.

ch06_error_plot.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

import math
import matplotlib.pyplot as plt

n_values = list(range(1, 31))
errors = [
    abs((1 + 1 / n)**n - math.e)
    for n in n_values
]

plt.plot(n_values, errors, marker="o")
plt.xlabel("n")
plt.ylabel("absolute error")
plt.grid(alpha=0.3)
plt.show()

print(f"error for n=1: {errors[0]:.6f}")
print(f"error for n=30: {errors[-1]:.6f}")
```

Source: Keeping the Meaning of the Data.

ch06_prime_gaps.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

from math import isqrt

def prime_gaps(limit):
    """Return (p, q, q-p) for consecutive primes q below limit."""
    if not isinstance(limit, int) or limit < 3:
        raise ValueError("limit must be an integer at least 3")

    primes = []
    for number in range(2, limit):
        if all(number % d for d in range(2, isqrt(number) + 1)):
            primes.append(number)

    return [
        (primes[i], primes[i + 1], primes[i + 1] - primes[i])
        for i in range(len(primes) - 1)
    ]

print(prime_gaps(12))

def test_prime_gaps():
    assert prime_gaps(3) == []
    assert prime_gaps(12) == [
        (2, 3, 1),
        (3, 5, 2),
        (5, 7, 2),
        (7, 11, 4),
    ]

    for p, q, gap in prime_gaps(1000):
        assert q - p == gap
        assert gap > 0

    try:
        prime_gaps(2)
    except ValueError:
        pass
    else:
        raise AssertionError("invalid input was accepted")

test_prime_gaps()
print("all prime-gap tests passed")
```

Source: From Data to an Honest Figure.

ch06_convergence.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

import math

def convergence_data(first_power=0, last_power=10):
    if type(first_power) is not int or type(last_power) is not int:
        raise ValueError("exponents must be integers")
    if first_power < 0 or last_power < first_power:
        raise ValueError("invalid exponent range")

    rows = []
    for exponent in range(first_power, last_power + 1):
        n = 2**exponent
        approximation = (1.0 + 1.0 / n)**n
        error = abs(math.e - approximation)
        rows.append({
            "n": n,
            "approximation": approximation,
            "error": error,
            "scaled_error": n * error,
        })
    return rows

data = convergence_data()
assert [row["n"] for row in data[:4]] == [1, 2, 4, 8]
assert all(row["error"] > 0 for row in data)
assert all(
    left["error"] > right["error"]
    for left, right in zip(data, data[1:])
)

for row in convergence_data(0, 3):
    print(
        f'n={row["n"]:2d}',
        f'approximation={row["approximation"]:.6f}',
        f'error={row["error"]:.6f}'
    )

def test_convergence_data():
    one = convergence_data(0, 0)
    assert len(one) == 1
    assert one[0]["n"] == 1
    assert math.isclose(one[0]["error"], math.e - 2)

    rows = convergence_data(4, 10)
    assert rows[-1]["n"] == 1024
    assert all(
        left["n"] * 2 == right["n"]
        for left, right in zip(rows, rows[1:])
    )
```

```
    assert abs(rows[-1]["scaled_error"] - math.e / 2) < .002

    try:
        convergence_data(5, 4)
    except ValueError:
        pass
    else:
        raise AssertionError("invalid range was accepted")

test_convergence_data()
print("all convergence-data tests passed")
```

Source: Saving and Reading Data.

ch06_csv.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

import csv
from pathlib import Path
from tempfile import TemporaryDirectory

rows = [(n, n * n) for n in range(1, 6)]
with TemporaryDirectory() as folder:
    path = Path(folder) / "squares.csv"
    with path.open(
        "w", newline="", encoding="utf-8"
    ) as stream:
        writer = csv.writer(stream)
        writer.writerow(["n", "square"])
        writer.writerows(rows)

    with path.open(
        newline="", encoding="utf-8"
    ) as stream:
        reader = csv.DictReader(stream)
        restored = [
            (int(row["n"]), int(row["square"]))
            for row in reader
        ]

assert restored == rows
print("restored:", restored)
print("CSV round trip passed")
```

6.2 Exercise Programs

ch06_examples.py

```
"""Printed listings from ch06. Run this complete file."""
import matplotlib
matplotlib.use("Agg")

import csv
from io import StringIO

def read_squares(stream):
    reader = csv.DictReader(stream)
    if reader.fieldnames != ["n", "square"]:
        raise ValueError("expected columns n,square")
    rows = []
    for number, row in enumerate(reader, start=2):
        if set(row) != {"n", "square"}:
            raise ValueError(f"bad fields on line {number}")
        try:
            n, square = int(row["n"]), int(row["square"])
        except (TypeError, ValueError) as exc:
            raise ValueError(f"bad integer on line {number}") from exc
        if square != n*n:
            raise ValueError(f"wrong square on line {number}")
        rows.append((n, square))
    return rows

assert read_squares(StringIO("n,square\n2,4\n3,9\n")) == [(2,4),(3,9)]
for bad in ["n,square\n3,8\n", "n,value\n3,9\n"]:
    try:
        read_squares(StringIO(bad))
    except ValueError:
        pass
    else:
        raise AssertionError("invalid CSV accepted")
def round_trip(rows):
    stream = StringIO(newline="")
    writer = csv.writer(stream)
    writer.writerow(["n", "square"])
    writer.writerows(rows)
    stream.seek(0)
    reader = csv.DictReader(stream)
    return [(int(r["n"]), int(r["square"])) for r in reader]

for original in [[(2, 4), (3, 9)], [(2, 4), (3, 8)]]:
    assert round_trip(original) == original
assert any(square != n*n for n, square in round_trip([(2, 4), (3, 8)]))

import math
import matplotlib.pyplot as plt

def convergence_data(ns):
    rows = []
    for n in ns:
        if type(n) is not int or n < 1:
            raise ValueError("positive integer n required")
        value = math.exp(n * math.log1p(1 / n))
        error = abs(math.e - value)
        rows.append({"n": n, "value": value,
                     "error": error, "scaled": n*error})
```

```
    return rows

def summarize(rows):
    if not rows:
        raise ValueError("nonempty data required")
    return {"count": len(rows),
            "largest_error": max(r["error"] for r in rows),
            "last_scaled_error": rows[-1]["scaled"]}

def plot_errors(rows):
    fig, axes = plt.subplots(1, 2, figsize=(9, 3.4))
    ns = [r["n"] for r in rows]
    errors = [r["error"] for r in rows]
    if any(e <= 0 for e in errors):
        raise ValueError("log plot requires positive errors")
    for ax in axes:
        ax.plot(ns, errors, ".", markersize=2)
        ax.plot(ns, [math.e/(2*n) for n in ns], "--")
        ax.set_xlabel("n")
        ax.set_ylabel("Absolute error")
    axes[1].set_xscale("log")
    axes[1].set_yscale("log")
    fig.tight_layout()
    return fig

rows = convergence_data(range(1, 201))
assert summarize(rows)["count"] == 200
print(summarize(rows))
for row in rows:
    if row["n"] in [1, 2, 10, 50, 100, 200]:
        print(row)
fig = plot_errors(rows)
plt.close(fig)
```

Chapter 7
Sequences and Linear Algebra

7.1 Main-Book Programs

Source: Terms and Recurrences.

ch07_recurrence.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

def recurrence_terms(a0, a1, count):
    if count < 0:
        raise ValueError("count must be nonnegative")
    if count == 0:
        return []
    if count == 1:
        return [a0]

    terms = [a0, a1]
    while len(terms) < count:
        terms.append(3 * terms[-1] - 2 * terms[-2])
    return terms

print(recurrence_terms(2, 5, 12))
```

Source: Three Views of a Sequence.

ch07_modular_period.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

def fibonacci_period(modulus):
```

```
    if modulus < 2:
        raise ValueError("modulus must be at least 2")

    a, b = 0, 1
    for period in range(1, modulus * modulus + 1):
        a, b = b, (a + b) % modulus
        if (a, b) == (0, 1):
            return period

    raise AssertionError("initial state did not return")

for m in range(2, 13):
    print(m, fibonacci_period(m))
```

Source: Three Views of a Sequence.

ch07_generating_function.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

import sympy as sp

x = sp.symbols("x")
F = x / (1 - x - x**2)

print(sp.series(F, x, 0, 12))
```

Source: Linear Algebra: Exact and Numerical.

ch07_exact_rank.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

import sympy as sp

A = sp.Matrix([
    [1, 2, 1],
    [2, 5, 3],
    [1, 3, 2]
])

print("determinant:", A.det())
print("rank:", A.rank())
print("rref:", A.rref())
# Independent lower and upper bounds for the rank.
assert A[:2, :2].det() == 1
assert A * sp.Matrix([1, -1, 1]) == sp.zeros(3, 1)
```

Source: Linear Algebra: Exact and Numerical.

ch07_linear_system.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

import sympy as sp

A = sp.Matrix([[1, 2, 1], [2, 5, 3], [1, 3, 2]])
b = sp.Matrix([1, 2, 1])
x0 = sp.Matrix([1, 0, 0])
v = sp.Matrix([1, -1, 1])
t = sp.symbols("t")
assert A * x0 == b
assert A * v == sp.zeros(3, 1)
assert A[:2, :2].det() == 1
assert A * (x0 + t * v) == b
print("all rational solutions:", list(x0 + t * v))

bad_b = sp.Matrix([1, 2, 2])
y = sp.Matrix([1, -1, 1])
assert y.T * A == sp.zeros(1, 3)
assert (y.T * bad_b)[0] == 1
print("inconsistent right-hand side:", list(bad_b))
print("certificate: 0 = 1 would be required")
```

Source: Linear Algebra: Exact and Numerical.

ch07_vandermonde.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

import sympy as sp

t = sp.symbols("t")
A = sp.Matrix([
    [1, t, t**2],
    [1, 1, 1],
    [1, 2, 4]
])

determinant = sp.factor(A.det())
print(determinant)
```

Source: Linear Algebra: Exact and Numerical.

ch07_coordinates.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

import sympy as sp

T = sp.Matrix([[1, 1], [1, 0]])
P = sp.Matrix([[1, 1], [0, 1]])
C = P.inv() * T * P
assert T * P == P * C
u = sp.Matrix([3, 2])
print("standard vector:", list(P * u))
print("updated, first route:", list(T * (P * u)))
print("updated, second route:", list(P * (C * u)))
```

Source: When Numerical Rank Changes.

ch07_numerical_rank.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

from fractions import Fraction
import numpy as np

assert Fraction(1, 10**16) != 0
print("eps    stored   s_max   s_min    tol     rank")
for text in ["1e-8", "1e-12", "1e-15", "1e-16"]:
    epsilon = float(text)
    matrix = np.array([[1.0, 1.0], [1.0, 1.0 + epsilon]])
    singular = np.linalg.svd(matrix, compute_uv=False)
    tolerance = singular[0] * 2 * np.finfo(float).eps
    stored = matrix[1, 1] - matrix[0, 1]
    rank = np.linalg.matrix_rank(matrix)
    print(f"{text:<6} {stored:.1e} {singular[0]:.1e}",
          f"{singular[1]:.1e} {tolerance:.1e} {rank}")
```

7.2 Exercise Programs

ch07_examples.py

```
"""Printed listings from ch07. Run this complete file."""

def recurrence_terms(count):
    if type(count) is not int or count < 0:
        raise ValueError("expected integer >= 0")
    a, b = 2, 5
    terms = []
    for _ in range(count):
        terms.append(a)
        a, b = b, 3*b - 2*a
    return terms

assert recurrence_terms(0) == []
assert recurrence_terms(1) == [2]
assert recurrence_terms(6) == [2, 5, 11, 23, 47, 95]
assert recurrence_terms(6) == [3*2**n - 1 for n in range(6)]

a = [1, 4]
while len(a) < 15:
    a.append(5*a[-1] - 6*a[-2])
assert a == [2*3**n - 2**n for n in range(15)]

def fibonacci_period(m):
    if type(m) is not int or m <= 0:
        raise ValueError("positive integer modulus required")
    a, b = 0, 1 % m
    initial = (a, b)
    for period in range(1, m*m + 1):
        a, b = b, (a+b) % m
        if (a, b) == initial:
            return period
    raise AssertionError("state must return")

periods = {m: fibonacci_period(m) for m in (2, 3, 5, 10)}
print(periods)
assert periods == {2: 3, 3: 8, 5: 20, 10: 60}

states = [(0, 1)]
for _ in range(periods[2]):
    a, b = states[-1]
    states.append((b, (a+b) % 2))
assert states == [(0, 1), (1, 1), (1, 0), (0, 1)]
assert states[0] not in states[1:-1]

import sympy as sp
x = sp.symbols('x')
A = (1-x) / (1-5*x+6*x*x)
series = sp.series(A, x, 0, 15).removeO().expand()
coefficients = [series.coeff(x, n) for n in range(15)]
assert coefficients == [2*3**n - 2**n for n in range(15)]

import numpy as np
for epsilon in [1e-8, 1e-12, 1e-15, 1e-16]:
    matrix = np.array([[1., 1.], [1., 1. + epsilon]])
    singular = np.linalg.svd(matrix, compute_uv=False)
    ranks = [int(np.count_nonzero(singular > tol))
             for tol in [1e-10, 1e-14]]
```

Chapter 8
Dynamics and Randomness

8.1 Main-Book Programs

Source: Randomness and Seeds.

ch08_random_walk.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

import random

def random_walk(steps, seed):
    if type(steps) is not int or steps < 0:
        raise ValueError("steps: expected integer >= 0")
    rng = random.Random(seed)
    position = 0
    path = [position]
    for _ in range(steps):
        position += rng.choice([-1, 1])
        path.append(position)
    return path

path = random_walk(40, seed=17)
print("endpoint, minimum, maximum:",
      path[-1], min(path), max(path))

from math import sqrt
from statistics import mean, pstdev

endpoints = [
    random_walk(400, seed=s)[-1] for s in range(5000)
]
print("sample mean:", round(mean(endpoints), 3))
print("empirical standard deviation:",
      round(pstdev(endpoints), 3))
print("theoretical standard deviation:", sqrt(400))
print("proportion with |endpoint| <= 40:",
```

```
      sum(abs(x) <= 40 for x in endpoints)
      / len(endpoints))

for size in [100, 1000, 5000]:
    sample = endpoints[:size]
    print(f"walks={size:4d}",
          f"mean={mean(sample):.3f}",
          f"empirical sd={pstdev(sample):.3f}")
```

Source: The Logistic Map.

ch08_logistic.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

def logistic_orbit(r, x0=0.2, burn_in=200, keep=20):
    import math
    if not (math.isfinite(r) and math.isfinite(x0)):
        raise ValueError("r and x0 must be finite")
    if not (0 <= r <= 4 and 0 <= x0 <= 1):
        raise ValueError("r in [0,4]; x0 in [0,1]")
    if any(type(n) is not int or n < 0
           for n in (burn_in, keep)):
        raise ValueError("counts: expected integers >= 0")
    x = x0
    for _ in range(burn_in):
        x = r * x * (1 - x)
    orbit = []
    for _ in range(keep):
        x = r * x * (1 - x)
        orbit.append(x)
    return orbit

for r in [2.8, 3.2, 3.5, 3.9]:
    values = logistic_orbit(r)
    print("r =", r)
    last_eight = values[-8:]
    for start in [0, 4]:
        block = last_eight[start:start + 4]
        print([round(x, 6) for x in block])

import matplotlib.pyplot as plt

parameters, samples = [], []
for j in range(1200):
    r = 2.8 + 1.2 * j / 1199
    orbit = logistic_orbit(r, burn_in=800, keep=80)
    parameters.extend([r] * len(orbit))
    samples.extend(orbit)
plt.plot(parameters, samples, ",k", alpha=0.25)
plt.xlabel("r")
plt.ylabel("retained orbit values")
plt.show()
```

8.2 Exercise Programs

ch08_examples.py

```
"""Printed listings from ch08. Run this complete file."""

import random

def random_walk(steps, seed):
    if type(steps) is not int or steps < 0:
        raise ValueError("nonnegative integer steps required")
    rng = random.Random(seed)
    path = [0]
    for _ in range(steps):
        path.append(path[-1] + rng.choice([-1, 1]))
    return path

def check_walk(path, steps):
    return (len(path) == steps + 1 and path[0] == 0
            and all(abs(b-a) == 1 for a, b in zip(path, path[1:]))
            and (path[-1] - steps) % 2 == 0
            and -steps <= path[-1] <= steps)

for n in [0, 1, 40, 100]:
    assert check_walk(random_walk(n, 17), n)
assert random_walk(0, 17) == [0]
assert random_walk(40, 17) == random_walk(100, 17)[:41]

import statistics
seeds = list(range(1000))
rows = []
for n in [100, 400, 1600]:
    endpoints = [random_walk(n, seed)[-1] for seed in seeds]
    rows.append((n, statistics.mean(endpoints),
                 statistics.stdev(endpoints), n**0.5))

x, y = 0.2, 0.2 + 1e-10
separation = [abs(x-y)]
for _ in range(100):
    x = 3.9*x*(1-x)
    y = 3.9*y*(1-y)
    separation.append(abs(x-y))
assert all(d >= 0 for d in separation)
```

Chapter 9
Graphs and Networks

9.1 Main-Book Programs

Source: Paths and Witnesses.

ch09_paths.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

import networkx as nx

G = nx.Graph()
G.add_nodes_from(range(6))
G.add_edges_from([
    (0, 1), (1, 2), (2, 3),
    (3, 4), (4, 5), (5, 0),
    (0, 3)
])

print("vertices:", G.number_of_nodes())
print("edges:", G.number_of_edges())
print("degrees:", dict(G.degree()))

path = nx.shortest_path(G, 1, 4)
print(path)

for u, v in zip(path, path[1:]):
    assert G.has_edge(u, v)
```

Source: Invariants and Optimization.

ch09_colouring.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

import networkx as nx

def find_coloring(G, k):
    vertices = sorted(G, key=G.degree, reverse=True)
    color = {}

    def extend(index):
        if index == len(vertices):
            return True

        vertex = vertices[index]
        forbidden = {color[w] for w in G[vertex] if w in color}

        for choice in range(k):
            if choice not in forbidden:
                color[vertex] = choice
                if extend(index + 1):
                    return True
                del color[vertex]

        return False

    return dict(color) if extend(0) else None

C5 = nx.cycle_graph(5)
print(find_coloring(C5, 3))
```

Source: Invariants and Optimization.

ch09_spanning_tree.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

from itertools import combinations
import networkx as nx

vertices = {0, 1, 2, 3}
edges = [(0, 1, 1), (1, 2, 2), (2, 3, 3),
         (0, 3, 8), (0, 2, 5), (1, 3, 6)]

def connected(chosen):
    reached = {0}
```

```
    while True:
        previous = set(reached)
        for u, v, weight in chosen:
            if u in reached or v in reached:
                reached.update([u, v])
        if reached == previous:
            return reached == vertices

totals = [
    sum(weight for u, v, weight in chosen)
    for chosen in combinations(edges, len(vertices) - 1)
    if connected(chosen)
]
G = nx.Graph()
G.add_nodes_from(vertices)
G.add_weighted_edges_from(edges)
T = nx.minimum_spanning_tree(G)
chosen = [(u, v, data["weight"])
          for u, v, data in T.edges(data=True)]
assert len(chosen) == len(vertices) - 1
assert all(G.has_edge(u, v) and G[u][v]["weight"] == w
           for u, v, w in chosen)
assert connected(chosen)
weight = sum(w for u, v, w in chosen)
assert len(totals) == 16
assert weight == min(totals) == 6
print("spanning trees checked:", len(totals))
print("minimum total weight:", weight)
print("chosen edges:", sorted((min(u, v), max(u, v))
                              for u, v, w in chosen))
```

Source: A Smallest Counterexample.

ch09_hamilton.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

from itertools import combinations, permutations
import networkx as nx

def labelled_graphs(n):
    possible = list(combinations(range(n), 2))
    for mask in range(1 << len(possible)):
        G = nx.Graph()
        G.add_nodes_from(range(n))
        G.add_edges_from(
            edge for bit, edge in enumerate(possible)
            if mask & (1 << bit)
        )
        yield G

def hamilton_cycle(G):
    n = len(G)
    if n < 3:
```

```
        return None
    for tail in permutations(range(1, n)):
        cycle = (0,) + tail
        if all(G.has_edge(cycle[i], cycle[(i + 1) % n])
               for i in range(n)):
            return cycle + (0,)
    return None

for n in range(1, 7):
    total = qualified = failures = 0
    first = None
    for G in labelled_graphs(n):
        total += 1
        if nx.is_connected(G) and min(dict(G.degree()).values()) >= 2:
            qualified += 1
            if hamilton_cycle(G) is None:
                failures += 1
                if first is None:
                    first = G.copy()
    assert total == 2 ** (n * (n - 1) // 2)
    print(n, total, qualified, failures)
    if first is not None:
        print(sorted(first.edges()))
        print(dict(first.degree()))
        break

left = {0, 1}
right = {2, 3, 4}

assert left | right == set(first)
assert left.isdisjoint(right)
assert all(
    (u in left and v in right) or (u in right and v in left)
    for u, v in first.edges()
)
assert len(left) != len(right)

print("part sizes:", len(left), len(right))
print("Hamilton cycle impossible:", len(left) != len(right))
```

9.2 Exercise Programs

ch09_examples.py

```
"""Printed listings from ch09. Run this complete file."""

def valid_path(graph, path, start, end):
    if not path or start not in graph or end not in graph:
        return False
    if path[0] != start or path[-1] != end:
        return False
    if any(v not in graph for v in path):
        return False
    if len(set(path)) != len(path):
```

```
        return False
    return all(v in graph[u] for u, v in zip(path, path[1:]))

graph = {0:{1}, 1:{0,2}, 2:{1}, 3:set()}
assert valid_path(graph, [0,1,2], 0, 2)
assert valid_path(graph, [3], 3, 3)
assert not valid_path(graph, [], 0, 2)
assert not valid_path(graph, [0,1], 0, 2)
assert not valid_path(graph, [0,2], 0, 2)

import random
from itertools import combinations
import networkx as nx
rng = random.Random(314159)
rows = []
for trial in range(10):
    G = nx.Graph()
    G.add_nodes_from(range(8))
    G.add_edges_from(e for e in combinations(range(8), 2)
                     if rng.random() < 0.35)
    degree_sum = sum(d for _, d in G.degree())
    assert degree_sum == 2*G.number_of_edges()
    rows.append((trial, G.number_of_edges(), degree_sum))

G = nx.cycle_graph(6)
H = nx.disjoint_union(nx.cycle_graph(3), nx.cycle_graph(3))
assert sorted(dict(G.degree()).values()) == sorted(dict(H.degree()).values())
assert not nx.is_isomorphic(G, H)

from itertools import product

def find_colouring(n, k):
    for colors in product(range(k), repeat=n):
        if all(colors[i] != colors[(i+1) % n] for i in range(n)):
            return colors
    return None

assert [next(k for k in range(1, 4)
             if find_colouring(n, k) is not None)
        for n in range(3, 11)] == [3,2,3,2,3,2,3,2]

from collections import Counter
edges = list(combinations(range(4), 2))
graphs = [tuple(e for e, bit in zip(edges, bits) if bit)
          for bits in product([0, 1], repeat=6)]
assert len(graphs) == len(set(graphs)) == 64
assert Counter(map(len, graphs)) == {0:1,1:6,2:15,3:20,4:15,5:6,6:1}

examples = [
 [(0,1,1),(0,2,4),(0,3,3),(1,2,2),(1,3,5),(2,3,6)],
 [(0,1,2),(1,2,2),(2,3,2),(3,0,2),(0,2,5)],
 [(0,1,-2),(0,2,0),(1,2,1),(1,3,3),(2,3,-1)]]
for weighted_edges in examples:
    G = nx.Graph()
    G.add_weighted_edges_from(weighted_edges)
    weights = []
    for selected in combinations(weighted_edges, len(G)-1):
        H = nx.Graph()
        H.add_nodes_from(G)
        H.add_weighted_edges_from(selected)
```

```
        if nx.is_connected(H):
            weights.append(sum(w for u, v, w in selected))
    tree = nx.minimum_spanning_tree(G)
    reported = sum(data['weight'] for u, v, data in tree.edges(data=True))
    assert reported == min(weights)
```

Chapter 10
Search and Ordered Structures

10.1 Main-Book Programs

Source: Combinatorial Search.

ch10_enumeration.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

from itertools import combinations, permutations, product

print(list(combinations(range(5), 3)))
print(len(list(permutations(range(6)))))
print(len(list(product([0, 1], repeat=8))))
```

Source: Combinatorial Search.

ch10_permutations.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

def separated_permutations(n):
    answers = []

    def extend(prefix, unused):
        if not unused:
            answers.append(tuple(prefix))
            return

        for value in sorted(unused):
            if prefix and abs(prefix[-1] - value) == 1:
```

```
                continue
            prefix.append(value)
            extend(prefix, unused - {value})
            prefix.pop()

    extend([], set(range(n)))
    return answers

print(separated_permutations(4))
```

Source: Combinatorial Search.

ch10_coin_change.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

def change_count(total, coins):
    if type(total) is not int or total < 0:
        raise ValueError("total must be a nonnegative integer")
    coins = tuple(coins)
    if any(type(c) is not int or c <= 0 for c in coins):
        raise ValueError("coins must be positive integers")
    if len(set(coins)) != len(coins):
        raise ValueError("coin values must be distinct")
    ways = [0] * (total + 1)
    ways[0] = 1
    for coin in sorted(coins):
        for remaining in range(coin, total + 1):
            ways[remaining] += ways[remaining - coin]
    return ways[total]

assert change_count(0, ()) == 1
assert change_count(1, ()) == 0
print(change_count(10, (1, 5, 10)))
```

Source: Relations and Hasse Diagrams.

ch10_order_tools.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

def validate_order(elements, leq):
    if any(x is None for x in elements):
        return False, "None is reserved for absence"
    try:
        if len(set(elements)) != len(elements):
            return False, "repeated element"
    except TypeError:
```

```
        return False, "elements must be hashable"
    for x in elements:
        if not leq(x, x):
            return False, ("not reflexive", x)

    for x in elements:
        for y in elements:
            if leq(x, y) and leq(y, x) and x != y:
                return False, ("not antisymmetric", x, y)

    for x in elements:
        for y in elements:
            for z in elements:
                if (leq(x, y) and leq(y, z)
                        and not leq(x, z)):
                    witness = ("not transitive", x, y, z)
                    return False, witness

    return True, "valid order"

elements = [1, 2, 3, 6]
leq = lambda x, y: y % x == 0
print(validate_order(elements, leq))

def maximal_elements(elements, leq):
    return [
        x for x in elements
        if not any(x != y and leq(x, y) for y in elements)
    ]

def greatest_element(elements, leq):
    candidates = [
        x for x in elements
        if all(leq(y, x) for y in elements)
    ]
    return candidates[0] if candidates else None

elements = [1, 2, 3, 6]
leq = lambda x, y: y % x == 0
print("maximal:", maximal_elements(elements, leq))
print("greatest:", greatest_element(elements, leq))

def covers(elements, leq):
    answer = []
    for x in elements:
        for y in elements:
            if x == y or not leq(x, y):
                continue
            between = any(
                z != x and z != y and leq(x, z) and leq(z, y)
                for z in elements
            )
            if not between:
                answer.append((x, y))
    return answer

elements = [1, 2, 3, 6]
leq = lambda x, y: y % x == 0
print(covers(elements, leq))
```

Source: Ideals and Lattices.

ch10_lattice.py

```
def analyze_lattice(elements, relation):
    """Analyze a nonempty finite ordered set given by its reflexive relation."""
    elements = tuple(elements)
    if not elements:
        raise ValueError("a lattice must have a nonempty universe")
    if any(x is None for x in elements):
        raise ValueError("None is reserved for an absent bound")
    relation = set(relation)

    if len(set(elements)) != len(elements):
        raise ValueError("elements must be distinct")
    universe = set(elements)
    if any(not isinstance(pair, tuple) or len(pair) != 2 for pair in relation):
        raise ValueError("relation must consist of ordered pairs")
    if any(x not in universe or y not in universe for x, y in relation):
        raise ValueError("relation contains an element not in elements")

    def leq(x, y):
        return (x, y) in relation

    for x in elements:
        if not leq(x, x):
            raise ValueError(f"relation is not reflexive: ({x!r}, {x!r}) is missing")
    for x in elements:
        for y in elements:
            if x != y and leq(x, y) and leq(y, x):
                raise ValueError(f"relation is not antisymmetric: {x!r}, {y!r}")
    for x in elements:
        for y in elements:
            for z in elements:
                if leq(x, y) and leq(y, z) and not leq(x, z):
                    raise ValueError(f"relation is not transitive: {x!r}, {y!r}, {z!r}")

    def least(items):
        candidates = [u for u in items if all(leq(u, v) for v in items)]
        return candidates[0] if len(candidates) == 1 else None

    def greatest(items):
        candidates = [u for u in items if all(leq(v, u) for v in items)]
        return candidates[0] if len(candidates) == 1 else None

    join, meet = {}, {}
    for x in elements:
        for y in elements:
            upper_bounds = [u for u in elements if leq(x, u) and leq(y, u)]
            lower_bounds = [l for l in elements if leq(l, x) and leq(l, y)]
            x_join_y = least(upper_bounds)
            x_meet_y = greatest(lower_bounds)
            if x_join_y is None or x_meet_y is None:
                return {
                    "is_lattice": False,
                    "pair": (x, y),
```

```
                    "common_upper_bounds": upper_bounds,
                    "common_lower_bounds": lower_bounds,
                    "missing": tuple(name for name, value in
                                     (("join", x_join_y), ("meet", x_meet_y))
                                     if value is None),
                }
            join[x, y] = x_join_y
            meet[x, y] = x_meet_y

    for x in elements:
        for y in elements:
            if meet[x, join[x, y]] != x or join[x, meet[x, y]] != x:
                raise AssertionError("absorption failed")

    return {
        "is_lattice": True,
        "elements": elements,
        "join_table": [[join[x, y] for y in elements] for x in elements],
        "meet_table": [[meet[x, y] for y in elements] for x in elements],
        "absorption_verified": True,
    }


def print_table(title, elements, rows):
    print(title)
    print("      " + "  ".join(str(x) for x in elements))
    for x, row in zip(elements, rows):
        print(f"{x:>3} : " + "  ".join(str(value) for value in row))


if __name__ == "__main__":
    # Example: the divisibility order on {1, 2, 3, 6}.
    elements = (1, 2, 3, 6)
    relation = {(x, y) for x in elements for y in elements if y % x == 0}
    result = analyze_lattice(elements, relation)

    if not result["is_lattice"]:
        print("This ordered set is not a lattice.")
        print("First failing pair:", result["pair"])
        print("Missing:", ", ".join(result["missing"]))
        print("Common upper bounds:", result["common_upper_bounds"])
        print("Common lower bounds:", result["common_lower_bounds"])
    else:
        print("Example: {1, 2, 3, 6}, ordered by divisibility.")
        print("This ordered set is a lattice.")
        print("2 join 3 =", result["join_table"][1][2])
        print("2 meet 3 =", result["meet_table"][1][2])
        print_table("Join table", result["elements"], result["join_table"])
        print_table("Meet table", result["elements"], result["meet_table"])
        print("Absorption verified.")
```

Source: Ideals and Lattices.

ch10_distributivity.py

```
def check_distributivity(elements, relation):
    """Test distributivity on a nonempty finite lattice.

    ``relation`` is a collection of pairs (x, y), meaning x <= y.  If the
    relation is not a lattice, a ValueError is raised.  Otherwise the first
    failure of either distributive law is returned as a certificate.
    """
    elements = tuple(elements)
    if not elements:
        raise ValueError("a lattice must have a nonempty universe")
    if any(x is None for x in elements):
        raise ValueError("None is reserved for an absent bound")
    relation = set(relation)
    universe = set(elements)

    if len(universe) != len(elements):
        raise ValueError("elements must be distinct")
    if any(not isinstance(pair, tuple) or len(pair) != 2 for pair in relation):
        raise ValueError("relation must consist of ordered pairs")
    if any(x not in universe or y not in universe for x, y in relation):
        raise ValueError("relation contains an element not in elements")

    def leq(x, y):
        return (x, y) in relation

    # Verify that the supplied relation is an order.
    for x in elements:
        if not leq(x, x):
            raise ValueError("the relation is not reflexive")
    for x in elements:
        for y in elements:
            if x != y and leq(x, y) and leq(y, x):
                raise ValueError("the relation is not antisymmetric")
            for z in elements:
                if leq(x, y) and leq(y, z) and not leq(x, z):
                    raise ValueError("the relation is not transitive")

    def join_of(x, y):
        upper = [u for u in elements if leq(x, u) and leq(y, u)]
        least = [u for u in upper if all(leq(u, v) for v in upper)]
        return least[0] if len(least) == 1 else None

    def meet_of(x, y):
        lower = [u for u in elements if leq(u, x) and leq(u, y)]
        greatest = [u for u in lower if all(leq(v, u) for v in lower)]
        return greatest[0] if len(greatest) == 1 else None

    join = {(x, y): join_of(x, y) for x in elements for y in elements}
    meet = {(x, y): meet_of(x, y) for x in elements for y in elements}
    if any(value is None for value in join.values()) or any(value is None for value in meet.
     values()):
        raise ValueError("the ordered set is not a lattice")
```

```
    for x in elements:
        for y in elements:
            for z in elements:
                left = meet[x, join[y, z]]
                right = join[meet[x, y], meet[x, z]]
                if left != right:
                    return {
                        "distributive": False,
                        "law": "x meet (y join z) = (x meet y) join (x meet z)",
                        "triple": (x, y, z),
                        "left_side": left,
                        "right_side": right,
                    }

                left = join[x, meet[y, z]]
                right = meet[join[x, y], join[x, z]]
                if left != right:
                    return {
                        "distributive": False,
                        "law": "x join (y meet z) = (x join y) meet (x join z)",
                        "triple": (x, y, z),
                        "left_side": left,
                        "right_side": right,
                    }

    return {"distributive": True}


if __name__ == "__main__":
    # The diamond lattice M_3: 0 below a, b, c, all below 1.
    elements = ("0", "a", "b", "c", "1")
    relation = {
        ("0", "0"), ("a", "a"), ("b", "b"), ("c", "c"), ("1", "1"),
        ("0", "a"), ("0", "b"), ("0", "c"), ("0", "1"),
        ("a", "1"), ("b", "1"), ("c", "1"),
    }

    result = check_distributivity(elements, relation)
    if result["distributive"]:
        print("The lattice is distributive.")
    else:
        x, y, z = result["triple"]
        print("The lattice M_3 is not distributive.")
        print("First failing law:", result["law"])
        print(f"For x = {x}, y = {y}, z = {z}:")
        print("left side =", result["left_side"])
        print("right side =", result["right_side"])
```

Source: Ideals and Distributivity.

ch10_ideal_lattice.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

P = ("p", "q", "r", "s")
relations = {
    ("p", "p"), ("q", "q"), ("r", "r"), ("s", "s"),
    ("p", "r"), ("q", "r"),
    ("p", "s"), ("q", "s"), ("r", "s")
}

def leq(x, y):
    return (x, y) in relations

def ideals(elements):
    answer = []
    for mask in range(1 << len(elements)):
        I = {x for bit, x in enumerate(elements) if mask & (1 << bit)}
        if all(x in I for y in I for x in elements if leq(x, y)):
            answer.append(frozenset(I))
    return answer

J = ideals(P)
for I in sorted(J, key=lambda x: (len(x), sorted(x))):
    print(sorted(I))

for I in J:
    for K in J:
        assert I | K in J
        assert I & K in J
        for L in J:
            assert I & (K | L) == (I & K) | (I & L)
            assert I | (K & L) == (I | K) & (I | L)

print(len(J), "ideals; both distributive laws verified")

irreducibles = [
    I for I in J if I and not any(
        K | L == I
        for K in J for L in J if K < I and L < I
    )
]
principal = {
    p: frozenset(x for x in P if leq(x, p))
    for p in P
}
assert set(irreducibles) == set(principal.values())
assert all(leq(p, q) == (principal[p] <= principal[q])
           for p in P for q in P)
ordered = sorted(irreducibles,
                 key=lambda I: (len(I), sorted(I)))
for I in ordered:
```

```
    print(sorted(I))
print("recovered order agrees on all 16 pairs")
```

Source: Ideals and Distributivity.

ch10_m3.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

M3 = ("0", "a", "b", "c", "1")
atoms = {"a", "b", "c"}

def join(x, y):
    if x == "0": return y
    if y == "0": return x
    if x == y: return x
    if x == "1" or y == "1": return "1"
    return "1"

def meet(x, y):
    if x == "1": return y
    if y == "1": return x
    if x == y: return x
    if x == "0" or y == "0": return "0"
    return "0"

def first_distributive_failure():
    for x in M3:
        for y in M3:
            for z in M3:
                left = meet(x, join(y, z))
                right = join(meet(x, y), meet(x, z))
                if left != right:
                    return (x, y, z), left, right
    return None

witness, left, right = first_distributive_failure()
print(witness, left, right)
```

10.2 Exercise Programs

ch10_examples.py

```
"""Printed listings from ch10. Run this complete file."""

from itertools import product, permutations, combinations

def separated(values):
    return all(abs(a-b) != 1 for a, b in zip(values, values[1:]))

prefix = (0, 2)
assert separated(prefix)
assert not separated((0, 1))
assert all(not separated(prefix + (v,)) for v in (1, 3))
assert not any(separated(prefix + tail)
               for tail in permutations((1, 3)))
assert {p for p in permutations(range(4)) if separated(p)} == {
    (1, 3, 0, 2), (2, 0, 3, 1)
}

def ideals(elements, leq):
    answers = []
    for bits in product([0, 1], repeat=len(elements)):
        I = frozenset(x for x, bit in zip(elements, bits) if bit)
        if all(x in I for y in I for x in elements if leq(x, y)):
            answers.append(I)
    return answers

chain_ideals = ideals([0,1,2], lambda x,y: x <= y)
assert set(chain_ideals) == {frozenset(), frozenset({0}),
                             frozenset({0,1}), frozenset({0,1,2})}

from math import factorial
for n in range(7):
    ps = list(permutations(range(n)))
    words = list(product([0, 1], repeat=n))
    subsets = [frozenset(c) for k in range(n+1)
               for c in combinations(range(n), k)]
    assert len(ps) == len(set(ps)) == factorial(n)
    assert len(subsets) == len(set(subsets)) == 2**n
    assert len(words) == len(set(words)) == 2**n

def no_three_ones(n):
    if type(n) is not int or n < 0:
        raise ValueError("expected integer >= 0")
    return [''.join(map(str, word))
            for word in product([0,1], repeat=n)
            if all(word[i:i+3] != (1,1,1) for i in range(n-2))]

counts = [len(no_three_ones(n)) for n in range(11)]
assert counts == [1,2,4,7,13,24,44,81,149,274,504]
assert all(counts[n] == sum(counts[n-3:n]) for n in range(3,11))

I = ideals([0,1,2], lambda x,y: x == y or (x == 0 and y in [1,2]))
assert len(I) == 5
for A, B, C in product(I, repeat=3):
    assert A & (B | C) == (A & B) | (A & C)
    assert A | (B & C) == (A | B) & (A | C)
```

Chapter 11
Testing and Proof

11.1 Main-Book Programs

Source: Testing Relationships.

ch11_gcd_invariant.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

import math

def gcd_with_checks(a, b):
    a, b = abs(a), abs(b)

    while b:
        old_gcd = math.gcd(a, b)
        new_a, new_b = b, a % b
        assert math.gcd(new_a, new_b) == old_gcd
        a, b = new_a, new_b

    return a

print(gcd_with_checks(-84, 30))
```

Source: Testing Relationships.

ch11_coloring_checker.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

graph = {0: {1}, 1: {0}, 2: set()}
```

```
def edge_only(colors):
    return all(colors[u] != colors[v]
               for u in graph for v in graph[u])

def valid_coloring(colors, k):
    if type(k) is not int or k < 0:
        return False
    if set(colors) != set(graph):
        return False
    if any(type(c) is not int or not 0 <= c < k
           for c in colors.values()):
        return False
    return all(colors[u] != colors[v]
               for u in graph for v in graph[u])

missing = {0: 0, 1: 1}
print("faulty checker accepts missing vertex:",
      edge_only(missing))
print("repaired checker accepts it:",
      valid_coloring(missing, 2))
assert valid_coloring({0: 0, 1: 1, 2: 0}, 2)
assert not valid_coloring(missing, 2)
assert not valid_coloring({0: 0, 1: 0, 2: 1}, 2)
assert not valid_coloring({0: 0, 1: 1, 2: 2}, 2)
assert not valid_coloring({0: 0, 1: 1, 2: 0, 3: 0}, 2)
assert not valid_coloring({0: False, 1: 1, 2: 0}, 2)
print("acceptance and rejection tests passed")
```

11.2 Exercise Programs

ch11_examples.py

```
"""Printed listings from ch11. Run this complete file."""

graph = {0:{1}, 1:{0}, 2:set()}

def valid_coloring(colors, k):
    if type(k) is not int or k < 0:
        return False
    if set(colors) != set(graph):
        return False
    if any(type(c) is not int or not 0 <= c < k
           for c in colors.values()):
        return False
    return all(colors[u] != colors[v]
               for u in graph for v in graph[u])

assert not valid_coloring({0:0, 1:1, 2:True}, 2)
assert valid_coloring({0:0, 1:1, 2:0}, 2)
assert valid_coloring({0:0, 1:1, 2:1}, 2)
assert not valid_coloring({0:0, 1:0, 2:0}, 2)
print("(0,0,0): rejected; equal colours on edge 0--1")
print("(0,1,0): accepted; (0,1,1): accepted")
```

```
from itertools import combinations, product, permutations
for n in range(8):
    U = frozenset(range(n))
    for k in range(n+1):
        source = {frozenset(c) for c in combinations(U, k)}
        target = {frozenset(c) for c in combinations(U, n-k)}
        assert {U-S for S in source} == target
        assert all(U-(U-S) == S for S in source)

import math

def bisect_checked(f, left, right, tolerance=1e-12):
    if not all(math.isfinite(x) for x in [left, right, tolerance]):
        raise ValueError("finite inputs required")
    if left >= right or tolerance <= 0:
        raise ValueError("ordered interval and positive tolerance required")
    def value(x):
        y = f(x)
        if not math.isfinite(y):
            raise ValueError("nonfinite function value")
        return y
    fl, fr = value(left), value(right)
    if fl == 0: return left, (left, left), "computed zero"
    if fr == 0: return right, (right, right), "computed zero"
    if (fl < 0) == (fr < 0):
        raise ValueError("opposite signs required")
    assert left < right and (fl < 0) != (fr < 0)
    while right-left > tolerance:
        mid = left/2 + right/2
        if mid == left or mid == right:
            raise ArithmeticError("requested width below resolution")
        fm = value(mid)
        if fm == 0: return mid, (mid, mid), "computed zero"
        if (fl < 0) == (fm < 0):
            left, fl = mid, fm
        else:
            right, fr = mid, fm
        assert left < right
        assert fl != 0 and fr != 0
        assert (fl < 0) != (fr < 0)
    return left/2+right/2, (left, right), "computed bracket"

q, (a,b), status = bisect_checked(lambda x:x*x-2, 1, 2)
assert b-a <= 1e-12
assert bisect_checked(lambda x:x-1, 1, 2)[2] == "computed zero"
q, (a,b), status = bisect_checked(lambda x:1e-200*(x*x-2), 1, 2)
assert a < b

def to_cycles(p):
    n = len(p)
    if any(type(x) is not int for x in p) or sorted(p) != list(range(n)):
        raise ValueError("expected permutation")
    seen, cycles = set(), []
    for start in range(n):
        if start in seen:
            continue
        cycle, x = [], start
        while x not in seen:
            cycle.append(x)
```

```
            seen.add(x)
            x = p[x]
        cycles.append(tuple(cycle))
    return cycles

def from_cycles(cycles, n):
    if type(n) is not int or n < 0:
        raise ValueError("expected size >= 0")
    flat = [x for cycle in cycles for x in cycle]
    if (any(not cycle for cycle in cycles)
        or any(type(x) is not int for x in flat)
        or sorted(flat) != list(range(n))):
        raise ValueError("cycles must partition the ground set")
    p = list(range(n))
    for cycle in cycles:
        for i, x in enumerate(cycle):
            p[x] = cycle[(i+1) % len(cycle)]
    return p

for n in range(8):
    for p in permutations(range(n)):
        cycles = to_cycles(p)
        assert from_cycles(cycles, n) == list(p)
        assert to_cycles(from_cycles(cycles, n)) == cycles

import sympy as sp
for A in [sp.Matrix([[1,2],[3,4]]), sp.Matrix([[1,2],[2,4]])]:
    B = A.copy()
    B.row_swap(0, 1)
    assert B.det() == -A.det()
    B.row_swap(0, 1)
    assert B == A

checked = 0
for n in range(2, 21):
    for x, y, z in product(range(n), repeat=3):
        assert ((x+y) % n + z) % n == (x + (y+z) % n) % n
        checked += 1
assert checked == 44099

valid = {0:0, 1:1, 2:0}
bad = [
    {0:0, 1:0, 2:0},       # equal colors on edge 0--1
    {0:0, 1:1},            # missing isolated vertex
    {0:0, 1:1, 2:2},       # outside palette
    {0:0, 1:1, 2:False},   # Boolean instead of integer
    {0:0, 1:1, 2:0, 3:0}   # extra vertex
]
assert valid_coloring(valid, 2)
assert all(not valid_coloring(c, 2) for c in bad)
assert valid_coloring({v:1-c for v,c in valid.items()}, 2)
```

Chapter 12
Reproducible Investigations

12.1 Main-Book Programs

Source: The Reproducible Record.

ch12_seeded_pairs.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

import json
import platform
import random
from pathlib import Path

def sample_pairs(count, bound, seed):
    rng = random.Random(seed)
    return [
        (rng.randint(1, bound), rng.randint(1, bound))
        for _ in range(count)
    ]

path = Path("seeded_pairs_record.json")
version = platform.python_version()
if not path.exists():
    record = {
        "count": 3, "bound": 10, "seed": 314159,
        "rule": "two independent randint(1, bound) calls",
        "python": version,
        "pairs": [[4, 5], [5, 2], [2, 10]],
    }
    path.write_text(json.dumps(record, indent=2), encoding="utf-8")

saved = json.loads(path.read_text(encoding="utf-8"))
expected = [tuple(pair) for pair in saved["pairs"]]
repeated = sample_pairs(saved["count"], saved["bound"], saved["seed"])
assert repeated == expected
print("saved sample reproduced:", repeated)
```

```
print("same Python version:", saved["python"] == version)
```

Source: Three-Element Operations.

ch12_operations.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

from collections import Counter
from itertools import product

ELEMENTS = range(3)
TRIPLES = tuple(product(ELEMENTS, repeat=3))

def apply(table, x, y):
    return table[3 * x + y]

def failure_witnesses(table):
    failures = []
    for x, y, z in TRIPLES:
        left = apply(table, apply(table, x, y), z)
        right = apply(table, x, apply(table, y, z))
        if left != right:
            failures.append((x, y, z))
    return failures

def direct_failure_count(table):
    count = 0
    for x, y, z in TRIPLES:
        xy = table[3 * x + y]
        yz = table[3 * y + z]
        if table[3 * xy + z] != table[3 * x + yz]:
            count += 1
    return count

addition_mod_3 = tuple(
    (x + y) % 3 for x in ELEMENTS for y in ELEMENTS
)
print(failure_witnesses(addition_mod_3))

def audit_all_operations():
    distribution = Counter()
    first_table = {}

    for table in product(ELEMENTS, repeat=9):
        failures = failure_witnesses(table)
        count = len(failures)
        assert count == direct_failure_count(table)
        distribution[count] += 1
        first_table.setdefault(count, table)

    assert sum(distribution.values()) == 3**9
    assert set(distribution) == set(range(28))
    return distribution, first_table
```

```
distribution, first_table = audit_all_operations()

left_projection = tuple(
    x for x in ELEMENTS for y in ELEMENTS
)
right_projection = tuple(
    y for x in ELEMENTS for y in ELEMENTS
)

assert failure_witnesses(left_projection) == []
assert failure_witnesses(right_projection) == []
assert failure_witnesses(addition_mod_3) == []

one_failure = (0, 0, 0, 0, 0, 0, 0, 1, 0)
assert failure_witnesses(one_failure) == [(2, 2, 1)]

print("one-failure table:", failure_witnesses(one_failure))

print("attained failure counts:", min(distribution),
      "through", max(distribution))
print("associative labelled tables:", distribution[0])
print("tables examined:", sum(distribution.values()))
```

Source: Part IV Capstone.

ch12_capstone.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

from collections import Counter
from itertools import product

S = (0, 1)
pairs = list(product(S, repeat=2))
triples = list(product(S, repeat=3))
frequency = Counter()
witnesses = {}
for table in product(S, repeat=4):
    failures = [
        (x, y, z) for x, y, z in triples
        if table[2 * table[2 * x + y] + z]
        != table[2 * x + table[2 * y + z]]
    ]
    operation = dict(zip(pairs, table))
    checked = [
        (x, y, z) for x, y, z in triples
        if operation[operation[x, y], z]
        != operation[x, operation[y, z]]
    ]
    assert failures == checked
    frequency[len(failures)] += 1
    witnesses.setdefault(len(failures), (table, failures))
```

```
assert sum(frequency.values()) == 16
assert dict(frequency) == {0: 8, 2: 4, 4: 2, 8: 2}
print("failure count, number of tables:",
      sorted(frequency.items()))
for count, (table, failures) in sorted(witnesses.items()):
    print("count:", count, "table:", table)
    print("failing triples:", failures)
```

12.2 Exercise Programs

ch12_examples.py

```
"""Printed listings from ch12. Run this complete file."""

from itertools import product, permutations
from collections import Counter

def failures(table, n):
    return [(x,y,z) for x,y,z in product(range(n), repeat=3)
            if table[n*table[n*x+y]+z] != table[n*x+table[n*y+z]]]

frequency, witnesses = Counter(), {}
for table in product(range(2), repeat=4):
    failed = failures(table, 2)
    operation = dict(zip(product(range(2), repeat=2), table))
    checked = [(x,y,z) for x,y,z in product(range(2), repeat=3)
               if operation[operation[x,y],z] != operation[x,operation[y,z]]]
    assert failed == checked
    frequency[len(failed)] += 1
    witnesses.setdefault(len(failed), (table, failed))
assert dict(frequency) == {0:8, 2:4, 4:2, 8:2}
assert sum(frequency.values()) == 16

import random

def recorded_walk(steps, seed):
    if type(steps) is not int or steps < 0:
        raise ValueError("nonnegative integer steps required")
    rng = random.Random(seed)
    path = [0]
    for _ in range(steps):
        path.append(path[-1] + rng.choice([-1,1]))
    return {"steps":steps, "seed":seed, "path":path,
            "endpoint":path[-1], "minimum":min(path),
            "maximum":max(path)}

first = recorded_walk(40, 17)
random.random()  # unrelated global activity
second = recorded_walk(40, 17)
assert first == second
assert (first["endpoint"], first["minimum"], first["maximum"]) == (4,-1,6)

from collections import defaultdict
```

```
def relabel(table, p):
    n = len(p)
    result = [None] * (n*n)
    for x,y in product(range(n), repeat=2):
        result[n*p[x]+p[y]] = p[table[n*x+y]]
    return tuple(result)

classes = defaultdict(list)
for table in product(range(3), repeat=9):
    if not failures(table, 3):
        canonical = min(relabel(table, p) for p in permutations(range(3)))
        classes[canonical].append(table)
assert sum(map(len, classes.values())) == 113
assert len(classes) == 24
assert Counter(map(len, classes.values())) == {1:2, 3:7, 6:15}
for representative, members in classes.items():
    orbit = {relabel(representative,p) for p in permutations(range(3))}
    assert set(members) == orbit

assert relabel((0,)*9, (1, 0, 2)) == (1,)*9
```

Appendix A
Python: An Optional Introduction

A.1 Main-Book Programs

Source: Getting Python Running.

appendix_a_first_run.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

print("Python is ready")
```

Source: Getting Python Running.

hello.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

message = "Hello, mathematics!"
print(message)
```

Source: Numbers, Names, and Calculations.

appendix__a__numbers.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

print(17 + 25)
print(9 * 8)
print(2 ** 10)
print(17 / 5)
print(17 // 5)
print(17 % 5)

length = 12
width = 8
area = length * width

print(area)

count = 1
count = count + 1

print(count)

n = 104729               # integer
x = 1.25                 # floating-point number
name = "Petersen"        # string
connected = True         # Boolean value
primes = [2, 3, 5, 7]    # list
edge = (0, 1)            # tuple
seen = {2, 3, 5}         # set
degree = {0: 3, 1: 2}    # dictionary

print(type(n).__name__)
print(type(primes).__name__)
print(type(degree).__name__)

from math import factorial

print(factorial(30))

print(0.1 + 0.2)
print((0.1 + 0.2) == 0.3)

from fractions import Fraction
import math

exact = Fraction(1, 10) + Fraction(2, 10)
close = math.isclose(0.1 + 0.2, 0.3)

print(exact)
print(close)
```

Source: Decisions and Repetition.

appendix_a_loops.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

number = 17

if number % 2 == 0:
    print("even")
else:
    print("odd")

n = -8

if n > 0:
    sign = 1
elif n < 0:
    sign = -1
else:
    sign = 0

print(sign)

for i in range(1, 6):
    print(i, i ** 2)

total = 0

for i in range(1, 11):
    total = total + i ** 2

print(total)

n = 1000
power = 1
exponent = 0

while power < n:
    power = 2 * power
    exponent = exponent + 1

print(exponent, power)

n = 60

for d in range(1, n + 1):
    if n % d == 0:
        print(d)
```

Source: Functions and Collections.

appendix_a_functions.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

def divisors(n):
    result = []

    for d in range(1, n + 1):
        if n % d == 0:
            result.append(d)

    return result


print(divisors(60))
print(divisors(13))

def divisors(n):
    if type(n) is not int or n <= 0:
        raise ValueError("n must be a positive integer")

    result = []
    for d in range(1, n + 1):
        if n % d == 0:
            result.append(d)
    return result


print(divisors(36))

primes = [2, 3, 5, 7, 11]

print(primes[0])
print(primes[-1])
print(primes[1:4])

primes.append(13)
print(len(primes))

squares = [n * n for n in range(1, 11)]
odd_squares = [x for x in squares if x % 2 == 1]

print(odd_squares)

edge = (3, 7)
u, v = edge

quotient, remainder = divmod(29, 6)

print(u, v)
print(quotient, remainder)
```

```
def rectangle_data(length, width):
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter


area, perimeter = rectangle_data(3, 7)
print(area, perimeter)

A = {1, 2, 3, 5}
B = {2, 4, 5}

print(A | B)          # union
print(A & B)          # intersection
print(A - B)          # difference
print(3 in A)

edges = [(0, 1), (1, 2), (2, 0), (2, 3)]
degree = {v: 0 for v in range(4)}

for u, v in edges:
    degree[u] = degree[u] + 1
    degree[v] = degree[v] + 1

print(degree)

for vertex, value in degree.items():
    print(vertex, value)

print(sum(degree.values()))

primes = [2, 3, 5, 7, 11]

for position, prime in enumerate(primes, start=1):
    print(position, prime)

gaps = []
for left, right in zip(primes, primes[1:]):
    gaps.append(right - left)

print("gaps:", gaps)

def is_prime(n):
    if type(n) is not int:
        raise ValueError("n must be an integer")
    if n < 2:
        return False

    divisor = 2
    while divisor * divisor <= n:
        if n % divisor == 0:
            return False
        divisor = divisor + 1

    return True

tests = [0, 1, 2, 49, 97, 91]

for n in tests:
    print(n, is_prime(n))
```

```
def power_residues(power, modulus):
    if type(power) is not int or power < 0:
        raise ValueError("power must be a nonnegative integer")
    if type(modulus) is not int or modulus <= 0:
        raise ValueError("modulus must be a positive integer")

    residues = set()
    for a in range(modulus):
        residues.add(pow(a, power, modulus))

    return sorted(residues)


print(power_residues(2, 8))
print(power_residues(3, 7))

def fibonacci(count):
    if type(count) is not int or count < 0:
        raise ValueError("count must be a nonnegative integer")
    if count == 0:
        return []
    if count == 1:
        return [0]

    terms = [0, 1]
    while len(terms) < count:
        terms.append(terms[-1] + terms[-2])

    return terms


values = fibonacci(15)
print(values)
print([x for x in values if x % 2 == 0])

def degree_report(vertices, edges):
    vertices = list(vertices)
    if len(set(vertices)) != len(vertices):
        raise ValueError("vertices must be distinct")
    degree = {v: 0 for v in vertices}
    seen = set()

    for u, v in edges:
        if u not in degree or v not in degree:
            raise ValueError("unknown vertex in an edge")
        if u == v:
            raise ValueError("loops are not permitted")
        edge = frozenset((u, v))
        if edge in seen:
            raise ValueError("repeated undirected edge")
        seen.add(edge)
        degree[u] = degree[u] + 1
        degree[v] = degree[v] + 1

    maximum = max(degree.values()) if degree else 0
    largest = [
        v for v in vertices if degree[v] == maximum
    ]
```

```
    return degree, largest


V = [0, 1, 2, 3]
E = [(0, 1), (1, 2), (2, 0), (2, 3)]

degree, largest = degree_report(V, E)
print(degree)
print("maximum-degree vertices:", largest)
print("degree sum:", sum(degree.values()))

def square_congruence_solutions(value, modulus):
    if type(modulus) is not int or modulus <= 0:
        raise ValueError("modulus must be a positive integer")

    solutions = []
    for x in range(modulus):
        if x * x % modulus == value % modulus:
            solutions.append(x)

    return solutions


solutions = square_congruence_solutions(1, 15)
print(solutions)

assert divisors(1) == [1]
assert divisors(13) == [1, 13]
assert divisors(36) == [1, 2, 3, 4, 6, 9, 12, 18, 36]

values = fibonacci(15)
assert len(values) == 15
assert all(
    values[i] == values[i - 1] + values[i - 2]
    for i in range(2, len(values))
)

print("all tests passed")
```

Source: Functions and Collections.

appendix__a__copying.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

original = [3, 1, 2]
alias = original
copy = original.copy()
alias.append(4)
copy.sort()
assert alias is original
assert copy is not original
print("original:", original)
print("alias:", alias)
```

```
print("copy:", copy)

rows = [[0, 0] for _ in range(2)]
rows[0][0] = 1
assert rows == [[1, 0], [0, 0]]
print("independent rows:", rows)
```

Source: Testing and Reading Generated Code.

appendix_a_exceptions.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

def square_sum(count):
    if type(count) is not int or count < 0:
        raise ValueError("count: expected integer >= 0")
    return sum(n * n for n in range(1, count + 1))

assert square_sum(0) == 0
assert square_sum(5) == 55
for bad in [-1, 2.5, True]:
    try:
        square_sum(bad)
    except ValueError:
        pass
    else:
        raise AssertionError("invalid input was accepted")
print("valid inputs checked;",
      "three invalid inputs rejected")
```

Source: Testing and Reading Generated Code.

appendix_a_module.py

```
"""Complete main-book program; continuations joined in reading order.
Generated by tools/synchronize.py.
"""

def square_sum(count):
    """Sum squares through count, an integer >= 0."""
    if type(count) is not int or count < 0:
        raise ValueError("count: expected integer >= 0")
    return sum(n * n for n in range(1, count + 1))

def main():
    assert square_sum(0) == 0
    assert square_sum(5) == 55
    print("sum of first five squares:", square_sum(5))

if __name__ == "__main__":
```

```
    main()
```

A.2 Exercise Programs

appa_examples.py

```
"""Printed listings from appa. Run this complete file."""

import math

def rectangle_data(length, width):
    for x in (length, width):
        if type(x) not in (int, float):
            raise ValueError("numeric lengths required")
        if x <= 0 or (type(x) is float and not math.isfinite(x)):
            raise ValueError("finite positive lengths required")
    return length*width, 2*(length+width)

assert rectangle_data(4, 4) == (16, 16)
assert rectangle_data(3, 7) == (21, 20)
for bad in [0, -2, float('nan'), float('inf'), True]:
    try:
        rectangle_data(bad, 3)
    except ValueError:
        pass
    else:
        raise AssertionError("invalid length accepted")

def sign(n):
    if type(n) is not int:
        raise ValueError("integer required")
    if n < 0:
        return -1
    if n == 0:
        return 0
    return 1

assert [sign(n) for n in [-7, 0, 12]] == [-1, 0, 1]

def sum_to(n):
    if type(n) is not int or n < 0:
        raise ValueError("nonnegative integer required")
    total = 0
    for k in range(1, n+1):
        total += k
    return total

for n in range(1001):
    assert sum_to(n) == n*(n+1)//2

from math import isqrt

def divisors(n):
    if type(n) is not int or n < 1:
```

```
        raise ValueError("positive integer required")
    small, large = [], []
    for d in range(1, isqrt(n)+1):
        if n % d == 0:
            small.append(d)
            if d*d != n:
                large.append(n//d)
    return small + large[::-1]

assert divisors(1) == [1]
assert divisors(9) == [1,3,9]
assert divisors(12) == [1,2,3,4,6,12]
assert divisors(36) == [1,2,3,4,6,9,12,18,36]
for n in range(1, 1001):
    assert divisors(n) == [d for d in range(1,n+1) if n%d == 0]

def triangular_formula(n):
    if type(n) is not int or n < 0:
        raise ValueError("nonnegative integer required")
    return [k*(k+1)//2 for k in range(1,n+1)]

def triangular_loop(n):
    if type(n) is not int or n < 0:
        raise ValueError("nonnegative integer required")
    result, total = [], 0
    for k in range(1,n+1):
        total += k
        result.append(total)
    return result

assert triangular_loop(5) == [1,3,6,10,15]
assert all(triangular_formula(n) == triangular_loop(n) for n in range(101))

from itertools import product

def relation_report(elements, relation):
    X = list(elements)
    if len(set(X)) != len(X):
        raise ValueError("distinct elements required")
    R = set(relation)
    if any(len(pair) != 2 or pair[0] not in X or pair[1] not in X
           for pair in R):
        raise ValueError("relation must consist of pairs on X")
    return {
      "reflexivity": next(((x,) for x in X if (x,x) not in R), None),
      "symmetry": next(((x,y) for x,y in product(X,repeat=2)
                        if (x,y) in R and (y,x) not in R), None),
      "antisymmetry": next(((x,y) for x,y in product(X,repeat=2)
                            if x != y and (x,y) in R and (y,x) in R), None),
      "transitivity": next(((x,y,z) for x,y,z in product(X,repeat=3)
                            if (x,y) in R and (y,z) in R and (x,z) not in R), None)
    }

X = [0,1,2]
R = {(0,0),(1,1),(2,2),(0,1),(1,2)}
r = relation_report(X, R)
assert r == {"reflexivity":None, "symmetry":(0,1),
             "antisymmetry":None, "transitivity":(0,1,2)}
assert relation_report([0,1], {(0,1),(1,0)})["antisymmetry"] == (0,1)
assert relation_report([0], set())["reflexivity"] == (0,)
```

```
def degree_report(vertices, edges):
    vertices = list(vertices)
    edges = list(edges)  # Preserve a one-pass iterator.
    if len(set(vertices)) != len(vertices):
        raise ValueError("vertices must be distinct")
    degree = {v: 0 for v in vertices}
    seen = set()

    for u, v in edges:
        if u not in degree or v not in degree:
            raise ValueError("unknown vertex in an edge")
        if u == v:
            raise ValueError("loops are not permitted")
        edge = frozenset((u, v))
        if edge in seen:
            raise ValueError("repeated undirected edge")
        seen.add(edge)
        degree[u] = degree[u] + 1
        degree[v] = degree[v] + 1

    assert sum(degree.values()) == 2 * len(edges)
    maximum = max(degree.values()) if degree else 0
    largest = [v for v in vertices if degree[v] == maximum]
    return degree, largest

# Isolated vertices remain in the degree dictionary.
assert degree_report([0, 1, 2, 3], [(0, 1), (1, 2)]) == (
    {0: 1, 1: 2, 2: 1, 3: 0}, [1]
)
# Retain all vertices tied for maximum degree.
assert degree_report([0, 1, 2], [(0, 1), (1, 2), (2, 0)]) == (
    {0: 2, 1: 2, 2: 2}, [0, 1, 2]
)
assert degree_report([], []) == ({}, [])
assert degree_report([0, 1], []) == ({0: 0, 1: 0}, [0, 1])
# The assertion must also work with a one-pass edge iterator.
edge_iterator = iter([(0, 1), (1, 2)])
assert degree_report([0, 1, 2], edge_iterator) == (
    {0: 1, 1: 2, 2: 1}, [1]
)

def expect_degree_error(vertices, edges):
    try:
        degree_report(vertices, edges)
    except ValueError:
        return
    raise AssertionError("invalid graph was accepted")

expect_degree_error([0, 1], [(0, 1), (1, 0)])
expect_degree_error([0, 1], iter([(0, 1), (1, 0)]))
expect_degree_error([0, 1], [(0, 1), (0, 1)])
expect_degree_error([0, 1], [(0, 2)])
expect_degree_error([0], [(0, 0)])
expect_degree_error([0, 0], [])
expect_degree_error([], [(0, 1)])

def fibonacci_mod(count, m):
    if type(count) is not int or count < 0 or type(m) is not int or m <= 0:
        raise ValueError("count >= 0 and modulus > 0 required")
```

```
    a, b, values = 0, 1 % m, []
    for _ in range(count):
        values.append(a)
        a, b = b, (a+b) % m
    return values

def first_repeated_pair(m):
    if type(m) is not int or m <= 0:
        raise ValueError("positive integer modulus required")
    a, b, index, seen = 0, 1 % m, 0, {}
    while (a,b) not in seen:
        seen[a,b] = index
        a,b = b,(a+b) % m
        index += 1
    return (a,b), seen[a,b], index

assert fibonacci_mod(8, 3) == [0,1,1,2,0,2,2,1]
assert first_repeated_pair(3) == ((0,1), 0, 8)
assert first_repeated_pair(1) == ((0,0), 0, 1)

def cube_solutions(a, m):
    if type(a) is not int or type(m) is not int or m <= 0:
        raise ValueError("integer a and positive integer m required")
    return [x for x in range(m) if pow(x, 3, m) == a % m]

assert cube_solutions(1, 7) == [1,2,4]
assert cube_solutions(6, 7) == [3,5,6]
assert cube_solutions(0, 9) == [0,3,6]
assert cube_solutions(1, 9) == [1,4,7]
assert cube_solutions(2, 9) == []
assert cube_solutions(5, 1) == [0]

def pythagorean_triples(bound):
    if type(bound) is not int or bound < 1:
        raise ValueError("positive integer bound required")
    return [(a,b,c) for c in range(1,bound+1)
            for a in range(1,c+1) for b in range(a,c+1)
            if a*a+b*b == c*c]

triples = pythagorean_triples(100)
assert len(triples) == len(set(triples)) == 52
assert (3,4,5) in triples and (60,80,100) in triples

def broken_sum(n):
    total = 0
    for k in range(1,n):
        total += k
    return total

assert broken_sum(0) == 0
assert broken_sum(1) != 1
assert sum_to(1) == 1  # repaired function from A.3

def divisors_expanded(n):
    if type(n) is not int or n < 1:
        raise ValueError("positive integer required")
    result = []
    for d in range(1,n+1):
        if n % d == 0:
            result.append(d)
```

```
    return result

for n in range(1,1001):
    compact = [d for d in range(1,n+1) if n % d == 0]
    assert compact == divisors_expanded(n) == divisors(n)

def is_prime_reference(n):
    if type(n) is not int:
        raise ValueError("integer required")
    return n >= 2 and all(n % d != 0 for d in range(2,n))

def is_prime(n):
    if type(n) is not int:
        raise ValueError("integer required")
    if n < 2:
        return False
    if n % 2 == 0:
        return n == 2
    return all(n % d != 0 for d in range(3,isqrt(n)+1,2))

assert all(is_prime(n) == is_prime_reference(n) for n in range(10001))
assert [is_prime(n) for n in [-3,0,1,2,3,4,9,25,49]] == [False,False,False,True,True,False,
    False,False,False]
for bad in [True, 3.0, "3"]:
    for function in [is_prime, is_prime_reference]:
        try:
            function(bad)
        except ValueError:
            pass
        else:
            raise AssertionError("invalid input accepted")

import random

def walk_report(steps, seed):
    if type(steps) is not int or steps < 0:
        raise ValueError("nonnegative integer steps required")
    rng = random.Random(seed)
    path = [0]
    for _ in range(steps):
        path.append(path[-1] + rng.choice([-1,1]))
    return {"seed":seed, "path":path, "endpoint":path[-1],
            "minimum":min(path), "maximum":max(path)}

reports = [walk_report(100, seed) for seed in range(10)]
assert all(len(row['path']) == 101 for row in reports)
assert reports == [walk_report(100, seed) for seed in range(10)]
```

Appendix B
Verification and Reproduction Utilities

B.1 Exercise Support

ch10_examples 2.py

```
"""Printed listings from ch10. Run this complete file."""

from itertools import product, permutations, combinations

def ideals(elements, leq):
    answers = []
    for bits in product([0, 1], repeat=len(elements)):
        I = frozenset(x for x, bit in zip(elements, bits) if bit)
        if all(x in I for y in I for x in elements if leq(x, y)):
            answers.append(I)
    return answers

chain_ideals = ideals([0,1,2], lambda x,y: x <= y)
assert set(chain_ideals) == {frozenset(), frozenset({0}),
                             frozenset({0,1}), frozenset({0,1,2})}

from math import factorial
for n in range(7):
    ps = list(permutations(range(n)))
    words = list(product([0, 1], repeat=n))
    subsets = [frozenset(c) for k in range(n+1)
               for c in combinations(range(n), k)]
    assert len(ps) == len(set(ps)) == factorial(n)
    assert len(subsets) == len(set(subsets)) == 2**n
    assert len(words) == len(set(words)) == 2**n

def no_three_ones(n):
    if type(n) is not int or n < 0:
        raise ValueError("expected integer >= 0")
    return [''.join(map(str, word))
            for word in product([0,1], repeat=n)
            if all(word[i:i+3] != (1,1,1) for i in range(n-2))]

counts = [len(no_three_ones(n)) for n in range(11)]
assert counts == [1,2,4,7,13,24,44,81,149,274,504]
assert all(counts[n] == sum(counts[n-3:n]) for n in range(3,11))
```

```
I = ideals([0,1,2], lambda x,y: x == y or (x == 0 and y in [1,2]))
assert len(I) == 5
for A, B, C in product(I, repeat=3):
    assert A & (B | C) == (A & B) | (A & C)
    assert A | (B & C) == (A | B) & (A | C)
```

divisor_tool.py

```
"""Part II capstone: certified divisor lists with a transparent reference."""
from math import isqrt
from pathlib import Path
import json

def divisor_data(n):
    if type(n) is not int or n<1:raise ValueError('positive integer required')
    small,large=[],[]
    for d in range(1,isqrt(n)+1):
        if n%d==0:
            small.append(d)
            if d*d!=n:large.append(n//d)
    ds=small+large[::-1]
    return {'n':n,'divisors':ds,'count':len(ds)}

def reference(n):
    if type(n) is not int or n<1:raise ValueError('positive integer required')
    return [d for d in range(1,n+1) if n%d==0]

def check():
    for n in range(1,10001):
        row=divisor_data(n)
        assert row['divisors']==reference(n)
        assert row['count']==len(row['divisors'])
        assert all(n%d==0 for d in row['divisors'])
    for bad in [0,-1,3.5,True]:
        try:divisor_data(bad)
        except ValueError:pass
        else:raise AssertionError('invalid input accepted')
    rows=[divisor_data(n) for n in range(1,64) if divisor_data(n)['count']==6]
    assert len(rows)==10
    target=Path(__file__).resolve().parents[1]/'results'/'divisor_tool_results.json'
    target.write_text(json.dumps(rows,indent=2))
    print('Divisor-tool checks passed for 1 through 10000.')
    for row in rows:print(row)

if __name__=='__main__':check()
```

make_tables.py

```
"""Build LaTeX tables from the retained verification record."""
from pathlib import Path
import json
ROOT=Path(__file__).resolve().parents[1];R=ROOT/'results'
r=json.loads((R/'verified_results.json').read_text())
def sci(x,d=6):
 if x==0:return '$0$'
 m,e=f'{x:.{d}e}'.split('e')
 return '$'+m+r'\times10^{'+str(int(e))+'}$'
def table(name,cols,head,rows,long=False):
 env='longtable' if long else 'tabular'
 lines=[] if long else [r'\begin{center}']
 lines += [r'\begin{'+env+'}{'+cols+'}',r'\toprule',' & '.join(head)+r'\\\midrule'+(r'\endhead'
     if long else '')]
 lines += [' & '.join(map(str,row))+r'\\' for row in rows]
 lines += [r'\bottomrule',r'\end{'+env+'}']
 if not long:lines += [r'\end{center}']
 (R/(name+'.tex')).write_text('\n'.join(lines)+'\n')
table('fibonacci_table','rrrr',['$n$','$F_n$','$F_{n-1}$','$F_n/F_{n-1}$'],[(n,a,b,f'{v:.9f}')
    for n,a,b,v in r['fibonacci_ratios']])
table('sqrt_table','rrr',['$x$','Direct','Rationalized'],[(sci(x,0),sci(a),sci(b)) for x,a,b in
     r['sqrt_cancellation']])
table('quadratic_table','rrr',['$B$','Direct small-root magnitude','Stable magnitude'],[('$10^{
    '+str(k)+'}$',sci(d),sci(s)) for k,a,d,s in r['quadratic_scan'] if k in
    [4,6,7,8,9,10,12,16]])
table('convergence_table','rrr',['$n$','$E_n$ (careful evaluation)','$nE_n$'],[(n,f'{eb:.12f}',
    f'{n*eb:.9f}') for n,a,ea,eb in r['convergence_selected']])
table('rank_table','rrrr',[r'$\varepsilon$',r'Computed $s_{\min}$',r'$\tau=10^{-10}$',r'$\tau
    =10^{-14}$'],[(sci(e,0),sci(sm,3),a,b) for e,stored,smax,sm,a,b in r['rank']])
p=r['walk40']['17'];(R/'walk40.tex').write_text('\\begin{gather*}\n'+', '.join(map(str,p[:21]))
    +r',\\'+'\n'+', '.join(map(str,p[21:]))+' .\n\\end{gather*}\n')
table('walk40_summary','rrrr',['Seed','Endpoint','Minimum','Maximum'],[(s,p[-1],min(p),max(p))
    for s,p in r['walk40'].items()])
table('walk_statistics','rrrr',['Steps','Sample mean','Sample standard deviation',r'$\sqrt n$'
    ],[(n,f'{m:.3f}',f'{s:.6f}',f'{t:.0f}') for n,m,s,t in r['walk_statistics']])
table('handshaking','rrr',['Trial','$|E|$',r'$\sum_v\deg v$'],r['handshaking'])
table('cycles','rrrr',['Cycle',r'$\chi$','Two-colour assignments','Three-colour assignments'
    ],[(f'$C_{{{n}}}$',k,a,b) for n,k,a,b in r['cycles']])
table('one_failure','rrrrr',['$x$','$y$','$z$','$L$','$R$'],r['one_failure_check'],True)
rows=[]
for i,(t,size,aut) in enumerate(r['associative_classes'],1):
 text='$('+r';\ '.join(''.join(map(str,t[j:j+3])) for j in [0,3,6])+')$';rows.append((i,text,
    size,aut))
table('associative_classes','rlrr',['Class','Canonical table','Orbit size','Automorphisms'],
    rows,True)
p=r['pythagorean_primitive'];table('primitive_triples','rrrr',['Primitive triple','Multiples','
    Primitive triple','Multiples'],[('$'+str(tuple(p[i]))+'$',100//p[i][2],'$'+str(tuple(p[i
    +1]))+'$',100//p[i+1][2]) for i in range(0,16,2)])
table('walk100','rrrr',['Seed','Endpoint','Minimum','Maximum'],r['walk100'])
print('LaTeX tables generated from verified_results.json.')

# The submitted source is merged; update its embedded generated tables too.
import re
source = ROOT / 'Exercises_Help.tex'
text = source.read_text(encoding='utf-8')
count = 0
for table_path in sorted(R.glob('*.tex')):
```

```
    marker = 'results/' + table_path.name
    pattern = (r'(% BEGIN MERGED: ' + re.escape(marker) + r'\n).*?'
               r'(% END MERGED: ' + re.escape(marker) + r')')
    text, changed = re.subn(pattern,
        lambda match: match[1] + table_path.read_text(encoding='utf-8')
                      + '\n' + match[2], text, flags=re.S)
    count += changed
if not count:
    raise RuntimeError('No embedded result tables found in Exercises_Help.tex')
source.write_text(text, encoding='utf-8')
print(f'Synchronized {count} embedded result tables in {source.name}.')
```

ordered_set_analyzer.py

```
"""Finite ordered-set analyzer and exhaustive checks, standard library only."""
from itertools import product
from math import gcd, lcm
from pathlib import Path
import json

def analyze(elements, relation):
    X=list(elements); R=set(relation)
    if len(set(X)) != len(X): raise ValueError('distinct elements required')
    if any(len(p)!=2 or p[0] not in X or p[1] not in X for p in R):
        raise ValueError('relation must lie in X times X')
    for x in X:
        if (x,x) not in R: return {'valid':False,'property':'reflexivity','witness':[x]}
    for x,y in product(X,repeat=2):
        if x!=y and (x,y) in R and (y,x) in R:
            return {'valid':False,'property':'antisymmetry','witness':[x,y]}
    for x,y,z in product(X,repeat=3):
        if (x,y) in R and (y,z) in R and (x,z) not in R:
            return {'valid':False,'property':'transitivity','witness':[x,y,z]}
    leq=lambda x,y:(x,y) in R
    result={'valid':True,'elements':X}
    result['covers']=[(x,y) for x,y in product(X,repeat=2) if x!=y and leq(x,y)
      and not any(z not in (x,y) and leq(x,z) and leq(z,y) for z in X)]
    result['minimal']=[x for x in X if not any(y!=x and leq(y,x) for y in X)]
    result['maximal']=[x for x in X if not any(y!=x and leq(x,y) for y in X)]
    result['least']=[x for x in X if all(leq(x,y) for y in X)]
    result['greatest']=[x for x in X if all(leq(y,x) for y in X)]
    result['ideals']=[list(I) for bits in product([0,1],repeat=len(X))
        if all(not bits[j] or bits[i] or not leq(x,y)
               for i,x in enumerate(X) for j,y in enumerate(X))
        for I in [[x for x,b in zip(X,bits) if b]]]
    joins,meets,bad={}, {}, []
    for x,y in product(X,repeat=2):
        lower=[z for z in X if leq(z,x) and leq(z,y)]
        upper=[z for z in X if leq(x,z) and leq(y,z)]
        meet=[z for z in lower if all(leq(w,z) for w in lower)]
        join=[z for z in upper if all(leq(z,w) for w in upper)]
        if len(meet)!=1 or len(join)!=1:
            bad.append({'pair':[x,y],'lower_bounds':lower,'upper_bounds':upper,
                        'meet_candidates':meet,'join_candidates':join})
        if len(meet)==1:meets[x,y]=meet[0]
```

```
        if len(join)==1:joins[x,y]=join[0]
    # The contract uses nonempty lattices.
    result['lattice']=bool(X) and not bad
    result['failures']=bad
    if result['lattice']:
        result['meet_table']=[[meets[x,y] for y in X] for x in X]
        result['join_table']=[[joins[x,y] for y in X] for x in X]
    return result

def lattice_reference(X,R):
    # Independent extremal-bound test: unique maximal lower and minimal upper bounds.
    for x,y in product(X,repeat=2):
        L={z for z in X if (z,x) in R and (z,y) in R}
        U={z for z in X if (x,z) in R and (y,z) in R}
        maxL={z for z in L if not any(z!=w and (z,w) in R for w in L)}
        minU={z for z in U if not any(z!=w and (w,z) in R for w in U)}
        if len(maxL)!=1 or len(minU)!=1:return False
    return bool(X)

def check():
    empty = analyze([], set())
    assert empty['valid'] and not empty['lattice']
    assert empty['ideals'] == [[]]
    assert not lattice_reference([], set())
    singleton = analyze([0], {(0, 0)})
    assert singleton['lattice']
    counts=[]
    for n in range(1,5):
        X=list(range(n)); off=[(x,y) for x,y in product(X,repeat=2) if x!=y]
        valid=lattices=0
        for bits in product([0,1],repeat=len(off)):
            R={(x,x) for x in X}|{p for p,b in zip(off,bits) if b}
            report=analyze(X,R)
            if report['valid']:
                valid+=1;lattices+=report['lattice']
                assert report['lattice']==lattice_reference(X,R)
                # Duality tests for ideals and upper sets.
                dual=analyze(X,{(y,x) for x,y in R})
                ideals={frozenset(I) for I in report['ideals']}
                dual_ideals={frozenset(I) for I in dual['ideals']}
                universe=frozenset(X)
                assert {universe-I for I in ideals}==dual_ideals
                for I in ideals:
                    for J in ideals:
                        assert universe-(I|J)==(universe-I)&(universe-J)
                        assert universe-(I&J)==(universe-I)|(universe-J)
                direct_upper={frozenset(x for x,b in zip(X,bits) if b)
                  for bits in product([0,1],repeat=n)
                  if all(not bits[i] or bits[j] or (x,y) not in R
                    for i,x in enumerate(X) for j,y in enumerate(X))}
                assert direct_upper==dual_ideals
        counts.append((n,2**len(off),valid,lattices))
    assert [r[2] for r in counts]==[1,3,19,219]
    assert [r[3] for r in counts]==[1,2,6,36]
    X=[1,2,3,6];R={(a,b) for a,b in product(X,repeat=2) if b%a==0}
    divisor=analyze(X,R)
    assert divisor['meet_table']==[[gcd(a,b) for b in X] for a in X]
    assert divisor['join_table']==[[lcm(a,b) for b in X] for a in X]
    assert len(divisor['ideals'])==6
    invalid=analyze([0,1,2],{(0,0),(1,1),(2,2),(0,1),(1,2)})
```

```
    assert invalid['property']=='transitivity' and invalid['witness']==[0,1,2]
    V=analyze([0,1,2],{(0,0),(1,1),(2,2),(0,1),(0,2)})
    assert V['valid'] and not V['lattice']
    ideals=[frozenset(I) for I in V['ideals']]
    ideal_order=analyze(ideals,{(I,J) for I,J in product(ideals,repeat=2) if I<=J})
    assert ideal_order['lattice']
    out={'counts':counts,'divisors_of_6':divisor,'invalid_relation':invalid,'nonlattice':V}
    destination=Path(__file__).resolve().parents[1]/'results'/'ordered_set_results.json'
    destination.write_text(json.dumps(out,indent=2))
    print('Ordered-set exhaustive checks passed:',counts)

if __name__=='__main__':check()
```

prepare_examples.py

```
"""Extract every chapter's listings from the merged companion source.
Run from any directory; the source is Exercises_Help.tex beside programs/.
"""
from pathlib import Path
import hashlib
import json
import re

ROOT = Path(__file__).resolve().parents[1]
source = ROOT / 'Exercises_Help.tex'
text = source.read_text(encoding='utf-8')
chapters = re.findall(
    r'% BEGIN MERGED: chapters/(ch\d\d|appa)\.tex\n(.*?)'
    r'% END MERGED: chapters/\1\.tex', text, re.S)
expected = {f'ch{n:02}' for n in range(1, 13)} | {'appa'}
if {name for name, _ in chapters} != expected or len(chapters) != 13:
    raise RuntimeError('Expected exactly Chapters 1--12 and Appendix A')
record = {'source': source.name,
          'sha256': hashlib.sha256(source.read_bytes()).hexdigest(),
          'chapters': []}
for name, chapter in chapters:
    blocks = re.findall(r'\\begin\{lstlisting\}(.*?)\\end\{lstlisting\}',
                        chapter, re.S)
    if not blocks:
        raise RuntimeError(f'No Python listings found in {name}')
    header = f'"""Printed listings from {name}. Run this complete file."""\n'
    code = '\n\n'.join(block.strip() for block in blocks)
    if 'matplotlib' in code:
        header += 'import matplotlib\nmatplotlib.use("Agg")\n'
    output = ROOT / 'programs' / f'{name}_examples.py'
    output.write_text(header + '\n' + code + '\n', encoding='utf-8')
    record['chapters'].append({'file': output.name, 'listings': len(blocks)})
(ROOT / 'results').mkdir(exist_ok=True)
(ROOT / 'results' / 'listing_inventory.json').write_text(
    json.dumps(record, indent=2) + '\n', encoding='utf-8')
print(f'Extracted {sum(r["listings"] for r in record["chapters"])} listings '
      'into 13 complete chapter files.')
```

run_checks.py

```
"""Check the printed chapter programs and both capstones in fresh processes."""
from pathlib import Path
import hashlib
import json
import os
import subprocess
import sys
import time

ROOT = Path(__file__).resolve().parents[1]
if not __debug__:
    raise RuntimeError('Run without -O: assertions are part of the checks')
files = [ROOT / 'programs' / f'ch{n:02}_examples.py' for n in range(1, 13)]
files += [ROOT / 'programs' / name for name in
          ['appa_examples.py', 'divisor_tool.py', 'ordered_set_analyzer.py']]
records = []
for script in files:
    start = time.monotonic()
    done = subprocess.run([sys.executable, str(script)], cwd=ROOT,
                          env={**os.environ, 'MPLBACKEND': 'Agg'},
                          text=True, capture_output=True, timeout=300)
    records.append({'script': script.name, 'sha256': hashlib.sha256(script.read_bytes()).
    hexdigest(),
                    'python': sys.version, 'returncode': done.returncode,
                    'seconds': round(time.monotonic() - start, 3),
                    'stdout': done.stdout, 'stderr': done.stderr})
    print(script.name, 'PASS' if done.returncode == 0 else 'FAIL', flush=True)
(ROOT / 'results').mkdir(exist_ok=True)
(ROOT / 'results' / 'chapter_check_results.json').write_text(
    json.dumps(records, indent=2) + '\n', encoding='utf-8')
if any(r['returncode'] for r in records):
    raise SystemExit('A check failed; see results/chapter_check_results.json')
print('All 13 chapter programs and both capstones passed.')
```

verify_solutions.py

```
"""Reproduce the numerical tables and finite searches in the solutions.
Run from any directory: python3 programs/verify_solutions.py
Requires numpy, matplotlib, sympy, networkx. Never run with python -O.
"""
from pathlib import Path
from itertools import product, permutations, combinations
from collections import Counter, defaultdict
from fractions import Fraction
from decimal import Decimal, localcontext
import math, random, statistics, json, sys, platform
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import sympy as sp
import networkx as nx
```

```
ROOT = Path(__file__).resolve().parents[1]
OUT = ROOT / 'results'
OUT.mkdir(exist_ok=True)
FIG = ROOT / 'figures'
FIG.mkdir(exist_ok=True)
report = {'environment': {'python': sys.version, 'platform': platform.platform(),
 'numpy': np.__version__, 'matplotlib': matplotlib.__version__,
 'sympy': sp.__version__, 'networkx': nx.__version__}}

def is_prime_reference(n):
    if type(n) is not int: raise ValueError('expected integer')
    return n >= 2 and all(n % d for d in range(2, n))

def is_prime(n):
    if type(n) is not int: raise ValueError('expected integer')
    if n < 2: return False
    if n % 2 == 0: return n == 2
    return all(n % d for d in range(3, math.isqrt(n) + 1, 2))

def divisors(n):
    if type(n) is not int or n < 1: raise ValueError('positive integer required')
    low, high = [], []
    for d in range(1, math.isqrt(n) + 1):
        if n % d == 0:
            low.append(d)
            if d*d != n: high.append(n//d)
    return low + high[::-1]

G = nx.petersen_graph()
fig, axes = plt.subplots(1, 2, figsize=(8, 3.5))
for ax, positions, title in zip(axes, [nx.circular_layout(G), nx.spring_layout(G, seed=17)], ['
    Circular layout','Spring layout']):
    nx.draw_networkx(G, pos=positions, ax=ax, node_color='white', edgecolors='black', node_size
    =230, font_size=8)
    ax.set_title(title); ax.set_axis_off()
fig.tight_layout();fig.savefig(FIG/'petersen.pdf');plt.close(fig)

report['primes_below_200'] = [n for n in range(200) if is_prime(n)]
fib=[0,1]
for _ in range(19): fib.append(sum(fib[-2:]))
report['fibonacci_ratios'] = [(n, fib[n], fib[n-1], fib[n]/fib[n-1]) for n in range(5,21)]
report['six_divisors'] = [n for n in range(1,200) if len(divisors(n))==6][:10]
report['cube_residues'] = {str(m): [x**3 % m for x in range(m)] for m in [7,9]}
report['fermat_341'] = [11*31, pow(2,340,341)]
rows=[[n**4-n*n for n in range(7)]]
while len(rows[-1])>1: rows.append([b-a for a,b in zip(rows[-1],rows[-1][1:])])
report['differences']=rows
report['quadratic_prime_values']=[n*n+n+17 for n in range(17)]
report['mersenne']=[(p,2**p-1,sp.factorint(2**p-1)) for p in range(2,32) if is_prime(p)]
for n in range(1,10001): assert divisors(n)==[d for d in range(1,n+1) if n%d==0]
for n in range(10001): assert is_prime(n)==is_prime_reference(n)

def quad(B):
    t=2.0/B
    s=math.sqrt((1-t)*(1+t))
    large=(B/2)*(1+s)
    return large, (B-B*s)/2, 1/large
report['quadratic_scan']=[(k, *quad(10.0**k)) for k in range(1,17)]
```

```
report['sqrt_cancellation']=[(x, math.sqrt(1+x)-1, x/(math.sqrt(1+x)+1)) for x in [1e-4,1e-8,1e
    -12,1e-16,1e-20]]

def convergence_data(ns):
    return [(n, (1+1/n)**n, math.exp(n*math.log1p(1/n))) for n in ns]
conv=convergence_data(range(1,201))
report['convergence_selected']=[(n,a,math.e-a,math.e-b) for n,a,b in conv if n in
    [1,2,10,50,100,200]]
report['convergence_max_relative_change']=max(abs(a-b)/(math.e-b) for n,a,b in conv)
ns=np.array([x[0] for x in conv]); errors=np.array([math.e-x[2] for x in conv])
plt.rcParams.update({'font.size':10,'axes.spines.top':False,'axes.spines.right':False})
fig,ax=plt.subplots(1,2,figsize=(9,3.4))
for a in ax:
 a.plot(ns,errors,'.',ms=2,label='Computed error')
 a.plot(ns, math.e/(2*ns),'--',lw=1,label='e/(2n)')
 a.set_xlabel('n'); a.set_ylabel('Absolute error'); a.grid(alpha=.2)
ax[0].set_title('Linear axes'); ax[1].set_title('Logarithmic axes')
ax[1].set_xscale('log');ax[1].set_yscale('log'); ax[0].legend(fontsize=8)
fig.tight_layout(); fig.savefig(FIG/'convergence.pdf'); plt.close(fig)

report['large_n_convergence']=[]
for n in [10**4,10**8,10**12,10**16]:
 a=(1+1/n)**n; b=math.exp(n*math.log1p(1/n))
 with localcontext() as c:
  c.prec=80; dn=Decimal(n); exact=(1+1/dn)**n; exact_error=Decimal(1).exp()-exact
 report['large_n_convergence'].append((n,a,b,str(exact_error)))

x=sp.symbols('x')
a=[1,4]
for _ in range(13): a.append(5*a[-1]-6*a[-2])
coeff=[sp.expand(sp.series((1-x)/(1-5*x+6*x*x),x,0,15).removeO()).coeff(x,k) for k in range(15)
    ]
assert coeff==a==[2*3**n-2**n for n in range(15)]
report['recurrence_7_1']=a

def fib_period(m):
    if type(m) is not int or m<1: raise ValueError('positive integer required')
    a,b=0,1%m; initial=(a,b)
    for t in range(1,m*m+1):
        a,b=b,(a+b)%m
        if (a,b)==initial: return t
    raise AssertionError('invertible finite state update must return')
report['fibonacci_periods']={m:fib_period(m) for m in [1,2,3,5,10]}
report['rank'] = []
for eps in [1e-8,1e-12,1e-15,1e-16]:
 A=np.array([[1.,1.],[1.,1.+eps]])
 sv=np.linalg.svd(A,compute_uv=False)
 report['rank'].append((eps,A[1,1]-1,*sv,int(sum(sv>1e-10)),int(sum(sv>1e-14))))
T=sp.Matrix([[1,1],[1,0]]);P=sp.Matrix([[1,1],[0,1]]);C=sp.Matrix([[0,1],[1,1]])
assert T*P==P*C

def random_walk(steps,seed):
    if type(steps) is not int or steps<0: raise ValueError('nonnegative integer required')
    rng=random.Random(seed); path=[0]
    for _ in range(steps): path.append(path[-1]+rng.choice([-1,1]))
    return path
report['walk40']={s:random_walk(40,s) for s in [17,18]}
report['walk_statistics']=[(n,statistics.mean(random_walk(n,s)[-1] for s in range(1000)),
    statistics.stdev(random_walk(n,s)[-1] for s in range(1000)),math.sqrt(n)) for n in
    [100,400,1600]]
```

```
report['walk100_paths']={s:random_walk(100,s) for s in range(10)}
report['walk100']=[(s,random_walk(100,s)[-1],min(random_walk(100,s)),max(random_walk(100,s)))
    for s in range(10)]
for n in [0,1,40,100]:
 path=random_walk(n,17)
 assert len(path)==n+1 and path[0]==0 and all(abs(b-a)==1 for a,b in zip(path,path[1:])) and (
    path[-1]-n)%2==0
assert random_walk(40,17)==random_walk(100,17)[:41]

def orbit(r,initial=.2,burn=100,keep=100):
    value=initial; out=[]
    for i in range(burn+keep):
        value=r*value*(1-value)
        if i>=burn: out.append(value)
    return out
r=3.2
lo=(r+1-math.sqrt((r-3)*(r+1)))/(2*r)
hi=(r+1+math.sqrt((r-3)*(r+1)))/(2*r)
report['cycle']=[lo,hi,r*lo*(1-lo),r*hi*(1-hi)]
fig,axes=plt.subplots(2,2,figsize=(9,5.8),sharex=True,sharey=True)
for ax,(grid,burn) in zip(axes.ravel(),[(300,100),(1200,100),(300,1000),(1200,1000)]):
 for rr in np.linspace(2.8,4,grid):
  vals=orbit(rr,burn=burn,keep=100)
  ax.plot([rr]*len(vals),vals,',',color='black',alpha=.3)
 ax.set_title(f'{grid} parameters; burn-in {burn}')
 ax.set_xlabel('r'); ax.set_ylabel('Retained values')
fig.tight_layout();fig.savefig(FIG/'bifurcation.pdf',dpi=200);plt.close(fig)
v,w=.2,.2+1e-10; sep=[abs(v-w)]
for _ in range(100):v=3.9*v*(1-v); w=3.9*w*(1-w);sep.append(abs(v-w))
report['separation']=[(i,sep[i]) for i in [0,10,20,30,40,60,100]]
fig,ax=plt.subplots(figsize=(7.5,3));ax.semilogy(range(len(sep)),[d if d>0 else np.nan for d in
     sep]);ax.set_xlabel('Iteration');ax.set_ylabel('Absolute separation');ax.grid(alpha=.2);
    fig.tight_layout();fig.savefig(FIG/'separation.pdf');plt.close(fig)

rng=random.Random(314159)
handshake=[]
for i in range(10):
 G=nx.Graph();G.add_nodes_from(range(8));G.add_edges_from(e for e in combinations(range(8),2)
    if rng.random()<.35)
 deg=sum(dict(G.degree()).values());assert deg==2*G.number_of_edges();handshake.append((i,G.
    number_of_edges(),deg))
report['handshaking']=handshake

def colouring(vertices,edges,k):
    for colors in product(range(k),repeat=len(vertices)):
        d=dict(zip(vertices,colors))
        if all(d[u]!=d[v] for u,v in edges):return d
    return None
report['cycles']=[]
for n in range(3,11):
 edges=[(i,(i+1)%n) for i in range(n)]
 counts=[sum(all(c[u]!=c[v] for u,v in edges) for c in product(range(k),repeat=n)) for k in
    [2,3]]
 report['cycles'].append((n,2 if counts[0] else 3,*counts))

def all_graphs(n):
    edges=list(combinations(range(n),2))
    for bits in product([0,1],repeat=len(edges)):
        G=nx.Graph();G.add_nodes_from(range(n));G.add_edges_from(e for e,b in zip(edges,bits)
    if b);yield G
```

```
assert len(list(all_graphs(4)))==64
report['graphs4_edges']=dict(sorted(Counter(g.number_of_edges() for g in all_graphs(4)).items()
    ))
mst_inputs=[[(0,1,1),(0,2,4),(0,3,3),(1,2,2),(1,3,5),(2,3,6)],[(0,1,2),(1,2,2),(2,3,2),(3,0,2)
    ,(0,2,5)],[(0,1,-2),(0,2,0),(1,2,1),(1,3,3),(2,3,-1)]]
report['mst']=[]
for es in mst_inputs:
 G=nx.Graph();G.add_weighted_edges_from(es)
 candidates=[]
 for selected in combinations(es,len(G)-1):
  H=nx.Graph();H.add_nodes_from(G);H.add_weighted_edges_from(selected)
  if nx.is_tree(H):candidates.append(sum(w for _,_,w in selected))
 direct=min(candidates);tree=nx.minimum_spanning_tree(G);lib=sum(e[2]['weight'] for e in tree.
    edges(data=True));assert lib==direct
 report['mst'].append((es,direct,len(candidates)))
report['basic_counts']=[(n,len(list(permutations(range(n)))),sum(1 for mask in product([0,1],
    repeat=n)),2**n) for n in range(7)]
report['no111']=[sum('111' not in ''.join(map(str,w)) for w in product([0,1],repeat=n)) for n
    in range(11)]
for n in range(3,11):assert report['no111'][n]==sum(report['no111'][n-3:n])
D=divisors(30)
report['div30_covers']=[(a,b) for a in D for b in D if a!=b and b%a==0 and not any(c!=a and c!=
    b and c%a==0 and b%c==0 for c in D)]

def ideals(elements,leq):
    return [frozenset(x for x,b in zip(elements,bits) if b) for bits in product([0,1],repeat=
    len(elements)) if all(not bits[j] or bits[i] or not leq(x,y) for i,x in enumerate(elements)
     for j,y in enumerate(elements))]
I=ideals([0,1,2],lambda x,y:x==y or (x==0 and y in [1,2]))
assert len(I)==5
for A,B,C in product(I,repeat=3):
 assert A & (B|C)==(A&B)|(A&C) and A|(B&C)==(A|B)&(A|C)
for n in range(8):
 for k in range(n+1):
  U=set(range(n));a={frozenset(c) for c in combinations(U,k)};b={frozenset(c) for c in
    combinations(U,n-k)}
  assert {frozenset(U-c) for c in a}==b
report['addition_associativity_triples']=sum(n**3 for n in range(2,21))
for n in range(2,21):
 assert all(((x+y)%n+z)%n==(x+(y+z)%n)%n for x,y,z in product(range(n),repeat=3))

def failures(table,n):
    return [(x,y,z) for x,y,z in product(range(n),repeat=3) if table[n*table[n*x+y]+z]!=table[n
    *x+table[n*y+z]]]

def enumerate_operations(n):
    freq=Counter();witness={};associative=[]
    pairs=list(product(range(n),repeat=2));triples=list(product(range(n),repeat=3))
    for table in product(range(n),repeat=n*n):
        f=failures(table,n); op=dict(zip(pairs,table))
        direct=[(x,y,z) for x,y,z in triples if op[op[x,y],z]!=op[x,op[y,z]]]
        assert f==direct
        freq[len(f)]+=1;witness.setdefault(len(f),{'table':table,'failures':f})
        if not f:associative.append(table)
    assert sum(freq.values())==n**(n*n)
    return freq,witness,associative
f2,w2,a2=enumerate_operations(2);f3,w3,a3=enumerate_operations(3)
assert dict(f2)=={0:8,2:4,4:2,8:2} and len(a3)==113 and set(f3)==set(range(28))
report['two_operations']={'distribution':dict(sorted(f2.items())),'witnesses':w2}
report['three_operations']={'distribution':dict(sorted(f3.items())),'witnesses':w3}
```

```
def relabel(table,p):
    n=len(p);new=[None]*(n*n)
    for x,y in product(range(n),repeat=2):new[n*p[x]+p[y]]=p[table[n*x+y]]
    return tuple(new)
classes=defaultdict(list)
for table in a3:
 canon=min(relabel(table,p) for p in permutations(range(3)));classes[canon].append(table)
assert len(classes)==24
class_rows=[]
for canon,members in sorted(classes.items()):
 orbit_set={relabel(canon,p) for p in permutations(range(3))}
 assert set(members)==orbit_set
 aut=sum(relabel(canon,p)==canon for p in permutations(range(3)))
 assert len(members)*aut==6
 class_rows.append((canon,len(members),aut))
report['associative_classes']=class_rows
report['orbit_sizes']=dict(sorted(Counter(len(v) for v in classes.values()).items()))
t=(0,0,0,0,0,0,0,1,0)
report['one_failure_check']=[(x,y,z,t[3*t[3*x+y]+z],t[3*x+t[3*y+z]]) for x,y,z in product(range
    (3),repeat=3)]
report['pythagorean']=[(a,b,c) for c in range(1,101) for a in range(1,c+1) for b in range(a,c
    +1) if a*a+b*b==c*c]
assert len(report['pythagorean'])==52
report['pythagorean_primitive']=[t for t in report['pythagorean'] if math.gcd(*t)==1]

def triangle(n):return n*(n+1)//2
for n in range(1001):assert sum(range(n+1))==triangle(n)
report['fraction_third']=[str(Fraction(1,3)),str(Fraction(1/3)),str(Fraction(1/3)-Fraction(1,3)
    ),(1/3)*3]
report['passed']='All verification assertions passed.'
(OUT/'verified_results.json').write_text(json.dumps(report,indent=2,default=lambda o:int(o) if
    isinstance(o,sp.Integer) else str(o)))
print(json.dumps({k:v for k,v in report.items() if k not in ['three_operations','
    one_failure_check','environment']},indent=2,default=str))
print('All verification assertions passed.')
```

B.2 Collection and Build Tools

build_books.py

```
"""Compile the main book and both companions from the complete package.
Requires pdflatex, makeindex, and the LaTeX packages used in the sources.
Run tools/synchronize.py first; stale generated sources stop this build.
"""
from pathlib import Path
import shutil
import subprocess
import sys

HERE = Path(__file__).resolve().parents[1]
BOOK = HERE.parent
```

```
def main():
    for executable in ['pdflatex', 'makeindex']:
        if not shutil.which(executable):
            raise RuntimeError(f'Missing TeX executable: {executable}')
    subprocess.run([sys.executable, str(HERE/'tools/synchronize.py'), '--check'], check=True)
    for directory, stem, index in [
        (BOOK, 'Vibe_Math', True),
        (BOOK/'Solutions_Companion', 'Exercises_Help', False),
        (HERE, 'Python_Programs', False)]:
        if not (directory/(stem + '.tex')).is_file():
            raise FileNotFoundError(directory/(stem + '.tex'))
        # Remove old reference caches, but preserve source and production assets.
        for suffix in ['.aux', '.out', '.toc']:
            (directory/(stem + suffix)).unlink(missing_ok=True)
        for run in range(3):
            with (directory/f'build_pass{run+1}.txt').open('w') as log:
                subprocess.run(['pdflatex', '-interaction=nonstopmode',
                                '-halt-on-error', '-file-line-error', stem + '.tex'],
                               cwd=directory, stdout=log, stderr=subprocess.STDOUT,
                               check=True)
            if index and run < 2:
                subprocess.run(['makeindex', stem + '.idx'], cwd=directory, check=True)
        print('Built', directory/(stem + '.pdf'), flush=True)


if __name__ == '__main__':
    main()
```

check_book.py

```
"""Run all complete main-book programs and retain evidence in results/.
Each program runs in a fresh process and a temporary working directory.
Usage: python3 tools/check_book.py
"""
from pathlib import Path
from contextlib import redirect_stdout
from itertools import product
import hashlib
import importlib.metadata
import io
import json
import os
import platform
import runpy
import subprocess
import sys
import tempfile
import time

ROOT = Path(__file__).resolve().parents[1]
RESULTS = ROOT / 'results'
RESULTS.mkdir(exist_ok=True)

def environment():
    return {'python': sys.version, 'platform': platform.platform(),
```

```
            'packages': {name: importlib.metadata.version(name) for name in
                         ['numpy', 'sympy', 'networkx', 'matplotlib']}}

def worker(filename):
    source = ROOT / 'main_programs' / filename
    with tempfile.TemporaryDirectory(prefix='vibe_math_') as directory:
        os.chdir(directory)
        namespace = runpy.run_path(str(source), run_name='__main__')
        if filename == 'ch12_operations.py':
            distribution = namespace['distribution']
            first = namespace['first_table']
            witnesses = {}
            for count, table in sorted(first.items()):
                failed = namespace['failure_witnesses'](table)
                operation = dict(zip(product(range(3), repeat=2), table))
                independent = [(x, y, z) for x, y, z in product(range(3), repeat=3)
                    if operation[operation[x, y], z] != operation[x, operation[y, z]]]
                assert failed == independent and len(failed) == count
                witnesses[count] = {'table_row_major': table, 'failures': failed}
            assert sum(distribution.values()) == 19683
            assert distribution[0] == 113 and set(witnesses) == set(range(28))
            report = {'source': 'main_programs/' + filename,
                      'sha256': hashlib.sha256(source.read_bytes()).hexdigest(),
                      'environment': environment(),
                      'distribution': dict(sorted(distribution.items())),
                      'witnesses': witnesses}
            (RESULTS / 'three_operations.json').write_text(
                json.dumps(report, indent=2) + '\n', encoding='utf-8')

def boundaries():
    import networkx as nx
    with redirect_stdout(io.StringIO()):
        euler = runpy.run_path(str(ROOT / 'main_programs/ch01_euler.py'))['test_euler']
        lattice = runpy.run_path(str(ROOT / 'main_programs/ch10_lattice.py'))['analyze_lattice'
    ]
        distributive = runpy.run_path(
            str(ROOT / 'main_programs/ch10_distributivity.py'))['check_distributivity']
        assert euler('one vertex', nx.empty_graph(1)) == (1, 0, 1)
        assert euler('one edge', nx.path_graph(2)) == (2, 1, 1)
        assert euler('triangle', nx.cycle_graph(3)) == (3, 3, 2)
        assert euler('nonplanar', nx.complete_graph(5)) is None
        rejected = [nx.Graph(), nx.DiGraph([(0, 1)]),
                    nx.MultiGraph([(0, 1)]), nx.Graph([(0, 0)]),
                    nx.disjoint_union(nx.cycle_graph(3), nx.cycle_graph(3))]
        for graph in rejected:
            try:
                euler('invalid input', graph)
            except ValueError:
                pass
            else:
                raise AssertionError('Euler accepted an input outside its domain')
        for routine in [lattice, distributive]:
            for empty in [[], (), iter(())]:
                try:
                    routine(empty, set())
                except ValueError as error:
                    assert 'nonempty' in str(error)
                else:
                    raise AssertionError('Empty lattice universe was accepted')
            try:
```

```
                routine([None], {(None, None)})
            except ValueError as error:
                assert 'None' in str(error)
            else:
                raise AssertionError('None label was accepted')
        assert lattice([0], {(0, 0)})['is_lattice']
        assert distributive([0], {(0, 0)})['distributive']
        assert not lattice([0, 1], {(0, 0), (1, 1)})['is_lattice']
        elements = range(5)
        relation = {(x, y) for x in elements for y in elements
                    if x == y or x == 0 or y == 4}
        assert lattice(elements, relation)['is_lattice']
        witness = distributive(elements, relation)
        assert not witness['distributive']
        assert witness['left_side'] != witness['right_side']
    return {'Euler': 'connected planar cases, nonplanar case, and five exclusions passed',
            'lattices': 'empty-universe and None rejection, singleton, antichain, and M3
     witness passed'}

def saved_record_check():
    """Repeat the saved-record experiment in two genuinely fresh processes."""
    source = ROOT / 'main_programs/ch12_seeded_pairs.py'
    with tempfile.TemporaryDirectory(prefix='vibe_saved_record_') as directory:
        record_path = Path(directory) / 'seeded_pairs_record.json'
        outputs = []
        original = None
        for run in range(2):
            done = subprocess.run([sys.executable, str(source)], cwd=directory,
                                  text=True, capture_output=True, timeout=30)
            assert done.returncode == 0, done.stderr
            outputs.append(done.stdout)
            assert record_path.is_file(), 'Saved-record program created no record'
            content = record_path.read_bytes()
            if run == 0:
                original = content
            else:
                assert content == original, 'Second run rewrote the saved record'
        record = json.loads(original)
        assert record['pairs'] == [[4, 5], [5, 2], [2, 10]]
        assert record['count'] == 3 and record['bound'] == 10
        assert record['seed'] == 314159
        # A saved sample is evidence to test, not a value to overwrite silently.
        record['pairs'][0][0] += 1
        record_path.write_text(json.dumps(record), encoding='utf-8')
        changed = record_path.read_bytes()
        rejected = subprocess.run([sys.executable, str(source)], cwd=directory,
                                  text=True, capture_output=True, timeout=30)
        assert rejected.returncode != 0 and 'AssertionError' in rejected.stderr
        assert record_path.read_bytes() == changed
        return {'source': source.name, 'sha256': hashlib.sha256(source.read_bytes()).hexdigest
    (),
                'fresh_process_runs': 2, 'record_unchanged': True,
                'altered_sample_rejected': True, 'stdout': outputs,
                'saved_record': json.loads(original)}


if __name__ == '__main__':
    if not __debug__:
        raise RuntimeError('Run without -O: assertions are required')
    if len(sys.argv) == 3 and sys.argv[1] == '--worker':
```

```
        worker(sys.argv[2])
    else:
        files = sorted((ROOT / 'main_programs').glob('*.py'))
        manifest = json.loads((ROOT / 'program_manifest.json').read_text())
        expected = set(manifest['book_programs'])
        if {script.name for script in files} != expected:
            raise RuntimeError('Main programs differ from manifest; synchronize first')
        for script in files:
            if hashlib.sha256(script.read_bytes()).hexdigest() != manifest['book_programs'][
    script.name]['sha256']:
                raise RuntimeError('Source hash differs from manifest: ' + script.name)
        report = {'environment': environment(), 'programs': []}
        for script in files:
            start = time.monotonic()
            done = subprocess.run([sys.executable, str(Path(__file__).resolve()),
                                   '--worker', script.name],
                                  env={**os.environ, 'MPLBACKEND': 'Agg'},
                                  text=True, capture_output=True, timeout=300)
            report['programs'].append({
                'file': script.name, 'returncode': done.returncode,
                'sha256': hashlib.sha256(script.read_bytes()).hexdigest(),
                'seconds': round(time.monotonic() - start, 3),
                'stdout': done.stdout, 'stderr': done.stderr})
            print(script.name, 'PASS' if done.returncode == 0 else 'FAIL', flush=True)
        report['boundary_checks'] = boundaries()
        report['saved_record_check'] = saved_record_check()
        (RESULTS / 'book_checks.json').write_text(json.dumps(report, indent=2) + '\n')
        if any(p['returncode'] for p in report['programs']):
            raise SystemExit('A program failed; read results/book_checks.json')
        print(f'All {len(files)} main-book programs, boundary checks, and saved-record checks
    passed.')
```

synchronize.py

```
"""Synchronize the main programs, exercise snapshot, inventory, and catalogue.

Run from any directory in the complete manuscript package:
    python3 tools/synchronize.py
    python3 tools/synchronize.py --check
Edit Vibe_Math.tex, Solutions_Companion/Exercises_Help.tex, and the two
external lattice files. Generated files must not be edited independently.
"""
from pathlib import Path
import argparse
import ast
import hashlib
import json
import re
import shutil
import subprocess
import sys

HERE = Path(__file__).resolve().parents[1]
BOOK = HERE.parent
MAIN = BOOK / 'Vibe_Math.tex'
```

```
SOLUTIONS = BOOK / 'Solutions_Companion'
TITLES = ['Mathematics First', 'Arithmetic Investigations',
          'Conjectures and Tests', 'Trustworthy Mathematical Answers',
          'Exact and Approximate Methods', 'Pictures, Tables, and Evidence',
          'Sequences and Linear Algebra', 'Dynamics and Randomness',
          'Graphs and Networks', 'Search and Ordered Structures',
          'Testing and Proof', 'Reproducible Investigations']
HEADER = ('"""Complete main-book program; continuations joined in reading order.\n'
          'Generated by tools/synchronize.py.\n"""\n\n')
EXCLUDED = {'print("Hello)',
            'python3 -m pip install sympy numpy networkx matplotlib',
            'py -m pip install sympy numpy networkx matplotlib'}


def digest(data):
    return hashlib.sha256(data).hexdigest()


def json_text(value):
    return json.dumps(value, indent=2) + '\n'


def expand_source(path, stack=()):
    """Expand line-level include/input commands; diagnose missing files/cycles."""
    path = Path(path).resolve()
    if path in stack:
        raise ValueError('Cyclic TeX inclusion: ' + str(path))
    text = path.read_text(encoding='utf-8')
    pattern = r'^[ \t]*\\(?:include|input)\{([^}\n]+)\}[ \t]*(?:%[^\n]*)?$'

    def include(match):
        child = Path(match[1])
        if not child.suffix:
            child = child.with_suffix('.tex')
        return expand_source(path.parent / child, stack + (path,))

    return re.sub(pattern, include, text, flags=re.M)


def extract_programs(text):
    """Every executable box needs its own explicit filename declaration."""
    pattern = (
        r'(?P<label>Program:|Continue the same program:|New file:)'
        r'\s*\\texttt\{(?P<name>[^}]+)\}'
        r'|^% external-program:\s*(?P<external>\S+)\s*$'
        r'|\\begin\{codebox\}\s*\n(?P<code>.*?)\\end\{codebox\}'
    )
    programs, skipped = {}, []
    pending = None
    for match in re.finditer(pattern, text, re.S | re.M):
        line = text.count('\n', 0, match.start()) + 1
        if match['label']:
            if pending is not None:
                raise ValueError(f'Filename declaration without a listing before line {line}')
            pending = match['name'].replace(r'\_', '_')
            continue
        code = match['code']
        if code is not None and code.strip() in EXCLUDED:
            if pending is not None:
                raise ValueError(f'Non-Python teaching box has a program label at {line}')
```

```
            skipped.append({'line': line, 'text': code.strip()})
            continue
        filename = match['external'] or pending
        if filename is None:
            raise ValueError(f'Executable listing without filename at line {line}')
        if not re.fullmatch(r'[A-Za-z0-9][A-Za-z0-9_.-]*\.py', filename):
            raise ValueError(f'Unsafe or unsupported program filename: {filename}')
        sections = re.findall(r'\\section(?:\[[^]]*\])?\{([^}]+)\}',
                              text[:match.start()])
        section = sections[-1] if sections else 'Opening example'
        external = match['external'] is not None
        chapter = len(re.findall(r'^\\chapter(?:\[[^]]*\])?\{',
                                 text[:match.start()], re.M))
        record = programs.setdefault(filename, {
            'external': external, 'source_lines': [], 'section': section,
            'chapter': chapter, 'blocks': []})
        if record['external'] != external or (external and record['source_lines']):
            raise ValueError(f'Conflicting declarations for {filename}')
        record['source_lines'].append(line)
        if not external:
            record['blocks'].append(code.strip())
            pending = None
    if pending is not None:
        raise ValueError('Filename declaration without a final listing: ' + pending)
    if not programs:
        raise ValueError('No executable programs found')
    return programs, skipped


def program_source(record):
    return HEADER + '\n\n'.join(record['blocks']) + '\n'


def extract_exercises(solution_text):
    """Match prepare_examples.py, also allowing a nonmutating consistency check."""
    chapters = re.findall(
        r'% BEGIN MERGED: chapters/(ch\d\d|appa)\.tex\n(.*?)'
        r'% END MERGED: chapters/\1\.tex', solution_text, re.S)
    if len(chapters) != len(TITLES) + 1 or len(dict(chapters)) != len(chapters):
        raise ValueError('Exercise chapter markers are missing or duplicated')
    generated = {}
    for name, chapter in chapters:
        blocks = re.findall(r'\\begin\{lstlisting\}(.*?)\\end\{lstlisting\}',
                            chapter, re.S)
        if not blocks:
            raise ValueError('No exercise listings in ' + name)
        header = f'"""Printed listings from {name}. Run this complete file."""\n'
        code = '\n\n'.join(block.strip() for block in blocks)
        if 'matplotlib' in code:
            header += 'import matplotlib\nmatplotlib.use("Agg")\n'
        generated[name + '_examples.py'] = header + '\n' + code + '\n'
    return generated


def enumeration_items(text):
    """Read the first enumerate environment, retaining any nested item text."""
    start = re.search(r'\\begin\{enumerate\}(?:\[[^]]*\])?', text)
    if start is None:
        raise ValueError('Missing exercise enumeration')
    depth, item_start, items = 1, None, []
```

```
    tokens = r'\\begin\{enumerate\}(?:\[[^]]*\])?|\\end\{enumerate\}|\\item\b'
    for match in re.finditer(tokens, text[start.end():]):
        a, b = start.end() + match.start(), start.end() + match.end()
        if match[0].startswith(r'\begin'):
            depth += 1
        elif match[0].startswith(r'\end'):
            depth -= 1
            if depth == 0:
                if item_start is not None:
                    items.append(text[item_start:a].strip())
                return items, text[b:]
        elif depth == 1:
            if item_start is not None:
                items.append(text[item_start:a].strip())
            item_start = b
    raise ValueError('Unclosed exercise enumeration')


def exercise_inventory(text, solution_text):
    questions, checkpoints = [], []
    chunks = text.split(r'\subsection*{Mathematical Checkpoint}')[1:]
    if len(chunks) != len(TITLES):
        raise ValueError('A mathematical checkpoint is missing or duplicated')
    for number, chunk in enumerate(chunks, 1):
        checkpoint, exercises = chunk.split(r'\subsection*{Exercises}', 1)
        checkpoints.append({'chapter': number,
                            'book_statement_latex': checkpoint.strip()})
        items, _ = enumeration_items(exercises)
        questions.extend({'id': f'{number}.{i}', 'book_statement_latex': q}
                         for i, q in enumerate(items, 1))
    appendix = text.split(r'\chapter{Python: An Optional Introduction}', 1)[1]
    appendix = re.split(r'^\\chapter(?:\[[^]]*\])?\{', appendix, maxsplit=1,
                        flags=re.M)[0]
    remainder = appendix.split(r'\section{Exercises and Next Steps}', 1)[1]
    appendix_items = []
    while r'\begin{enumerate}' in remainder:
        items, remainder = enumeration_items(remainder)
        appendix_items.extend(items)
    questions.extend({'id': f'A.{i}', 'book_statement_latex': q}
                     for i, q in enumerate(appendix_items, 1))
    for name in [f'ch{n:02}' for n in range(1, len(TITLES)+1)] + ['appa']:
        chapter = solution_text.split(f'% BEGIN MERGED: chapters/{name}.tex', 1)[1]
        chapter = chapter.split(f'% END MERGED: chapters/{name}.tex', 1)[0]
        chapter_id = 'A' if name == 'appa' else str(int(name[2:]))
        answers = re.findall(r'^\\exercise\{(.*)\}$', chapter, re.M)
        matched = [q for q in questions if q['id'].split('.')[0] == chapter_id]
        if len(answers) != len(matched):
            raise ValueError(f'Exercise/solution count differs in chapter {chapter_id}')
        for question, heading in zip(matched, answers):
            question['companion_heading_latex'] = heading
        if chapter_id != 'A':
            answer = chapter.split(r'\checkpoint', 1)[1].split(r'\exercise{', 1)[0]
            checkpoints[int(chapter_id)-1]['companion_answer_latex'] = answer.strip()
    return {'total_exercises': len(questions), 'mathematical_checkpoints': len(checkpoints),
            'further_investigations': text.count(r'\subsection*{Further Investigation}'),
            'part_capstones': solution_text.count(r'\capstone{'),
            'book_sha256': digest(MAIN.read_bytes()),
            'expanded_book_sha256': digest(text.encode()),
            'solutions_sha256': digest(solution_text.encode()),
            'note': 'Numbering is checked automatically; mathematical correspondence '
```

```
                  'still requires editorial review.',
            'exercises': questions, 'checkpoints': checkpoints}


def latex(value):
    for a, b in [('&', r'\&'), ('_', r'\_'), ('#', r'\#'), ('%', r'\%')]:
        value = value.replace(a, b)
    return value


def catalogue(programs):
    intro = (HERE / 'tools/catalogue_frontmatter.tex').read_text(encoding='utf-8')
    lines = [intro.replace('@@MAIN_PROGRAMS@@', str(len(programs)))]
    for number, title in enumerate(TITLES + ['Python: An Optional Introduction'], 1):
        appendix = number == len(TITLES) + 1
        if appendix:
            lines.append(r'\appendix')
        lines += [r'\chapter{' + title + '}', r'\section{Main-Book Programs}']
        for filename, record in programs.items():
            if record['chapter'] == number:
                lines.append(r'\Needspace{10\baselineskip}\noindent Source: ' +
                             record['section'] + r'.\par')
                lines.append(r'\program{' + latex(filename) + '}{main_programs/' + filename + '
    }')
        lines.append(r'\section{Exercise Programs}')
        filename = 'appa_examples.py' if appendix else f'ch{number:02}_examples.py'
        lines.append(r'\program{' + latex(filename) + '}{exercises/programs/' + filename + '}')
    lines.append(r'\chapter{Verification and Reproduction Utilities}')
    for directory, heading, excluded in [
        ('exercises/programs', 'Exercise Support', re.compile(r'(ch\d\d|appa)_examples\.py')),
        ('tools', 'Collection and Build Tools', re.compile(r'^$'))]:
        lines.append(r'\section{' + heading + '}')
        for path in sorted((HERE / directory).glob('*.py')):
            if not excluded.fullmatch(path.name):
                lines.append(r'\program{' + latex(path.name) + '}{' + directory + '/' + path.
    name + '}')
    lines.append(r'\end{document}')
    return '\n\n'.join(lines) + '\n'


def assert_file(path, expected, errors):
    if not path.is_file() or path.read_bytes() != expected:
        errors.append(str(path.relative_to(BOOK)))


def synchronize(check=False):
    for path in [MAIN, SOLUTIONS/'Exercises_Help.tex',
                 SOLUTIONS/'programs/prepare_examples.py', HERE/'tools/catalogue_frontmatter.
    tex']:
        if not path.is_file():
            raise FileNotFoundError('Required source missing (check capitalization): ' + str(
    path))
    text = expand_source(MAIN)
    solution_text = (SOLUTIONS/'Exercises_Help.tex').read_text(encoding='utf-8')
    programs, skipped = extract_programs(text)
    exercises = extract_exercises(solution_text)
    inventory = exercise_inventory(text, solution_text)
    if any(not 1 <= record['chapter'] <= len(TITLES)+1
           for record in programs.values()):
        raise ValueError('A program could not be assigned to a book chapter')
```

```
    generated = {}
    for filename, record in programs.items():
        path = HERE/'main_programs'/filename
        code = path.read_text(encoding='utf-8') if record['external'] else program_source(
     record)
        ast.parse(code, filename=filename)
        generated[path] = code.encode()
    for filename, code in exercises.items():
        ast.parse(code, filename=filename)
        generated[SOLUTIONS/'programs'/filename] = code.encode()
    # Report stale unreferenced programs; never silently delete someone's file.
    extra = set((HERE/'main_programs').glob('*.py')) - set(generated)
    if extra:
        raise ValueError('Unreferenced main programs; review before removing: ' +
                         ', '.join(sorted(p.name for p in extra)))
    if check:
        errors = []
        for path, expected in generated.items():
            assert_file(path, expected, errors)
        for directory in ['programs', 'results', 'figures']:
            source, snapshot = SOLUTIONS/directory, HERE/'exercises'/directory
            for path in source.rglob('*'):
                if path.is_file() and '__pycache__' not in path.parts and path.suffix != '.pyc':

                    assert_file(snapshot/path.relative_to(source), path.read_bytes(), errors)
        for filename in ['Exercises_Help.tex', 'SNmono-mod.cls', 'requirements.txt']:
            assert_file(HERE/'exercises'/filename, (SOLUTIONS/filename).read_bytes(), errors)
        assert_file(HERE/'Python_Programs.tex', catalogue(programs).encode(), errors)
        for root in [SOLUTIONS, HERE/'exercises']:
            assert_file(root/'results/exercise_inventory.json', json_text(inventory).encode(),
     errors)
        manifest = json.loads((HERE/'program_manifest.json').read_text())
        expected_files = {p.relative_to(HERE).as_posix() for p in HERE.rglob('*.py')}
        if {r['path'] for r in manifest['files']} != expected_files:
            errors.append('program_manifest.json: Python file inventory')
        if set(manifest['book_programs']) != set(programs):
            errors.append('program_manifest.json: main-book inventory')
        for record in manifest['files']:
            path = HERE/record['path']
            if not path.is_file() or digest(path.read_bytes()) != record['sha256']:
                errors.append(record['path'] + ': manifest hash')
        if errors:
            raise RuntimeError('Out-of-date files; run synchronize.py:\n' + '\n'.join(errors))
        print(f'Synchronization check passed: {len(programs)} main programs, '
              f'{len(inventory["exercises"])} exercises, '
              f'{len(inventory["checkpoints"])} checkpoints.')
        return
    for path, data in generated.items():
        if path.parent == HERE/'main_programs' and programs[path.name]['external']:
            continue
        path.parent.mkdir(parents=True, exist_ok=True)
        path.write_bytes(data)
    subprocess.run([sys.executable, str(SOLUTIONS/'programs/prepare_examples.py')], check=True)
    (SOLUTIONS/'results/exercise_inventory.json').write_text(json_text(inventory), encoding='
     utf-8')
    for directory in ['programs', 'results', 'figures']:
        shutil.copytree(SOLUTIONS/directory, HERE/'exercises'/directory,
                        dirs_exist_ok=True, ignore=shutil.ignore_patterns('__pycache__', '*.pyc
     '))
    for filename in ['Exercises_Help.tex', 'SNmono-mod.cls', 'requirements.txt']:
```

```
        shutil.copy2(SOLUTIONS/filename, HERE/'exercises'/filename)
    shutil.copy2(BOOK/'SNmono-mod.cls', HERE/'SNmono-mod.cls')
    shutil.copy2(SOLUTIONS/'requirements.txt', HERE/'requirements.txt')
    (HERE/'Python_Programs.tex').write_text(catalogue(programs), encoding='utf-8')
    previous = json.loads((HERE/'program_manifest.json').read_text()) if (HERE/'
     program_manifest.json').exists() else {}
    for filename, record in programs.items():
        record.pop('blocks')
        record['sha256'] = digest((HERE/'main_programs'/filename).read_bytes())
    manifest = {'book_programs': programs, 'excluded_boxes': skipped,
                'book_sha256': digest(MAIN.read_bytes()),
                'solutions_sha256': digest(solution_text.encode()),
                'related_main_book_figures': previous.get('related_main_book_figures', {}),
                'note': 'Main-book programs are extracted in reading order. External '
                        'lattice routines are maintained directly. PDF catalogue '
                        'listings read the downloadable source files.',
                'files': [{'path': path.relative_to(HERE).as_posix(),
                           'sha256': digest(path.read_bytes())}
                          for path in sorted(HERE.rglob('*.py'))]}
    (HERE/'program_manifest.json').write_text(json_text(manifest), encoding='utf-8')
    print(f'Synchronized {len(programs)} main programs, {len(exercises)} exercise '
          f'chapter programs, and {len(manifest["files"])} Python files in total.')


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--check', action='store_true', help='check without changing files')
    args = parser.parse_args()
    try:
        synchronize(check=args.check)
    except (OSError, ValueError, RuntimeError, IndexError, subprocess.CalledProcessError) as
     error:
        parser.exit(1, f'Synchronization failed: {error}\n')


if __name__ == '__main__':
    main()
```

test_synchronization.py

```
"""Regression tests for extraction and source expansion (no files are modified)."""
from pathlib import Path
import tempfile
import unittest

from synchronize import expand_source, extract_programs, program_source


class SynchronizationTests(unittest.TestCase):
    def test_continuation_and_new_file_stay_separate(self):
        text = r'''Program: \texttt{first.py}
\begin{codebox}
x = 1
\end{codebox}
Continue the same program: \texttt{first.py}
```

```
\begin{codebox}
print(x)
\end{codebox}
New file: \texttt{hello.py}
\begin{codebox}
print("Hello")
\end{codebox}
'''
        programs, skipped = extract_programs(text)
        self.assertEqual(set(programs), {'first.py', 'hello.py'})
        self.assertIn('x = 1\n\nprint(x)', program_source(programs['first.py']))
        self.assertNotIn('Hello', program_source(programs['first.py']))
        self.assertIn('Hello', program_source(programs['hello.py']))
        self.assertEqual(skipped, [])

    def test_chapter_assignment_does_not_depend_on_filename(self):
        programs, _ = extract_programs(r'''\chapter{First Chapter}
New file: \texttt{arbitrary.py}
\begin{codebox}
print(1)
\end{codebox}
\chapter[Short Title]{Second Chapter}
New file: \texttt{another.py}
\begin{codebox}
print(2)
\end{codebox}''')
        self.assertEqual(programs['arbitrary.py']['chapter'], 1)
        self.assertEqual(programs['another.py']['chapter'], 2)

    def test_unnamed_listing_is_an_error(self):
        with self.assertRaisesRegex(ValueError, 'without filename'):
            extract_programs('\\begin{codebox}\nprint(1)\n\\end{codebox}')

    def test_unsafe_filename_is_an_error(self):
        with self.assertRaisesRegex(ValueError, 'Unsafe'):
            extract_programs(r'''Program: \texttt{../escape.py}
\begin{codebox}
print(1)
\end{codebox}''')

    def test_teaching_box_is_excluded(self):
        programs, skipped = extract_programs(r'''Program: \texttt{valid.py}
\begin{codebox}
print(1)
\end{codebox}
\begin{codebox}
print("Hello)
\end{codebox}''')
        self.assertEqual(set(programs), {'valid.py'})
        self.assertEqual(len(skipped), 1)

    def test_recursive_include_and_input(self):
        with tempfile.TemporaryDirectory() as directory:
            root = Path(directory)
            (root/'main.tex').write_text('Start\n  \\include{chapter} % note\nEnd\n')
            (root/'chapter.tex').write_text('Chapter\n\\input{section.tex}\n')
            (root/'section.tex').write_text('Section\n')
            result = expand_source(root/'main.tex')
            self.assertIn('Chapter', result)
            self.assertIn('Section', result)
```

```
            self.assertNotIn(r'\include', result)
            self.assertNotIn(r'\input', result)

    def test_inclusion_cycle_is_an_error(self):
        with tempfile.TemporaryDirectory() as directory:
            root = Path(directory)
            (root/'a.tex').write_text('\\input{b}\n')
            (root/'b.tex').write_text('\\include{a}\n')
            with self.assertRaisesRegex(ValueError, 'Cyclic'):
                expand_source(root/'a.tex')

    def test_external_file_is_not_extracted_as_inline(self):
        programs, _ = extract_programs('% external-program: lattice.py\n')
        self.assertTrue(programs['lattice.py']['external'])
        self.assertEqual(programs['lattice.py']['blocks'], [])


if __name__ == '__main__':
    unittest.main(verbosity=2)
```